

Python-pandas

1.pandas的了解

- 1.1 什么是pandas
- 1.2 为什么用pandas
- 1.3 如何使用pandas
- 1.4 何时使用pandas

2.pandas的介绍

1.Series

- 2.1 series介绍
- 2.2 series的创建
- 2.3 series的基本用法
- 2.4 Series作业

3.DataFrame

- 3.1 DataFrame的介绍
- 3.2 DataFrame创建
- 3.3 DataFrame基本用法
- 3.4 DataFrame作业

3.pandas的基本操作

- 3.1 索引操作
- 3.2 重新索引
- 3.3 增删改查操作
 - 3.3.1 增加操作
- 3.4 高级索引
- 3.5 练习

4.Pandas对齐运算

- 4.1 算术运算和数据对齐
- 4.2 使用填充值的算术方法
- 4.3 DataFrame和Series混合运算

5.Pandas的函数应用

5.1 apply和applymap

5.2 排序

5.3 唯一值和成员属性

5.4 处理缺失数据

6.Pandas-层级索引

7.Pandas统计计算和描述

8.Pandas入门总结

9.数据加载，存储与文件格式

9.1 读取文件格式文件 (熟悉)

9.1.1 读取

9.1.1.1 read_csv()

9.1.1.2 read_table()

9.1.1.3 read_fwf()

9.1.1.4 read_clipboard()

9.1.1.5 read_excel()

9.1.1.7 read_html()

9.1.1.8 Json

9.1.1.9 分块读取大文件

9.1.2 写入

9.2 二进制数据格式(了解)

HDF5格式

9.3 Web APIs交互(了解)

9.4 数据库交互(熟悉)

10.数据的清洗和准备

1.处理缺失数据

过滤/滤除缺失数据

DataFrame

观察参数

填充数据

2.移除重复数据

3.利用映射或函数转换数据(重点)

4.替换值

- 5.重命名轴索引
- 6.离散化和面元化分
- 7.检测和过滤异常值
- 8.排列和随机采样
- 9.字符串对象方法
- 10.正则表达式
- 11.pandas的矢量化字符串函数
- 12.总结
- 13.案例

11.数据规整

- 1.层次化索引
- 2.数据链接
- 3.数据合并(pd.merge)

按照索引进行链接:

处理重复列名

join合并 二

concat

- 4.重塑层次化索引

- 5.轴向旋转(了解)

12.数据分组和聚合

- 1.案例

13.时间序列

为什么要学习pandas中的时间序列

- 1.日期和数据数据及工具

字符串和datetime的相互转换

- 2.时间序列基础
- 3.生产一段时间范围
- 4.频率和日期偏移量
- 5.移动数据
- 6.重采样
- 7.时间序列案例

1.pandas的了解

1.1 什么是pandas

Pandas是一个在Python中运行的开源数据处理和分析库。它提供了灵活高效的数据结构，使您可以轻松地处理和分析Excel文件中的数据。

1.2 为什么用pandas

使用Pandas操作Excel可以更加方便地进行数据清洗、整理和分析。Pandas提供了丰富的函数和方法，使得对数据的操作变得更加简单高效，同时它也提供了强大的统计分析和可视化能力。

1.3 如何使用pandas

1. 安装panda

```
1 pip install pandas
```

2. 导入Pandas库

```
1 import pandas as pd
```

3. 操作pandas

3.1. DataFrame()

`pd.DataFrame` 是Pandas库中最重要的数据结构之一。它表示一个二维、标记列和行的数据结构，类似于Excel中的电子表格或数据库中的表。您可以通过将不同类型的数据（如列表、字典、数组等）传递给 `pd.DataFrame()` 函数来创建DataFrame对象。下面是一个示例：


```

1 import pandas as pd
2
3 # 创建DataFrame 括号里面不写数据表示创建一个空的Excel文件
4 data = {'姓名': ['张三', '李四', '王五'],
5         '年龄': [25, 30, 28],
6         '城市': ['北京', '上海', '广州']}
7 df = pd.DataFrame(data)
8
9 # 显示DataFrame
10 print(df)

```

3.2. to_excel("文件地址",index=索引是否被保存,index_col="设置列的id")

`to_excel()` 是Pandas库中DataFrame对象的一个方法，用于将DataFrame数据保存为Excel文件。

您可以使用 `to_excel()` 方法将DataFrame数据保存为Excel文件

```

1 # 保存为Excel文件
2 df.to_excel('data.xlsx', index=False)

```

3.3. set_index("设置的索引列",inplace="是否在原文件中修改")

`set_index()` 是Pandas库中DataFrame对象的一个方法，用于设置DataFrame的索引列。

在Pandas中，DataFrame中的索引是用于标识和引用行的标签。默认情况下，索引为整数序列，从0开始。但是，根据数据的特点，您可能需要将某一列作为索引，以便更方便地访问、筛选和操作数据。

```

1 # 将"姓名"列设置为索引
2 df.set_index('姓名', inplace=True)

```

1.4 何时使用pandas

您可以在任何时候使用Pandas来操作Excel文件，具体取决于您的需求和数据处理阶段。

2.pandas的介绍

在pandas中有两种最主要也是最重要的数据数据结构series和DataFrame

1.Series

2.1 series介绍

series是一种一维标记的数组对象,能够保存任何数据类型(int,float,str,object...)包含了数据标签,称为索引

- 类似列表 index=['名字','姓名','班级']
- 由数据和索引组成
 - 索引(index)往左,数据(values)往右
 - 索引是自动创建

index	element
0	龙仔
1	麻辣龙虾

龙仔

2.2 series的创建

创建series的方式有三种

1. 导入pandas

```
1 import pandas as pd
```

2. 列表方式

```
1 data = [1, 2, 3, 4, 5]
2 s = pd.Series(data)
3 print(s)
```

3. NumPy数组对象方式

```
1 import numpy as np
2
3 data = np.array([1, 2, 3, 4, 5])
4 s = pd.Series(data)
5 print(s)
```

int64和int32

int64 和 int32 都是整数类型的数据类型，但它们在存储空间和取值范围上存在一些区别。

首先，int64 指的是用 64 位二进制来存储整数类型的数据，而 int32 则使用 32 位二进制。这意味着 int64 使用更多的内存空间来存储整数值，而 int32 则使用较少的内存空间。

其次，int64 的取值范围更广，可以表示更大的整数值，而 int32 的取值范围相对较小。具体而言，int64 的取值范围为 -2^{63} 到 $2^{63}-1$ （含两端），而 int32 的取值范围为 -2^{31} 到 $2^{31}-1$ （含两端）。

在实际应用中，选择使用 int64 还是 int32 取决于数据的特征和占用内存的需求。如果处理的整数数据较大，并且内存资源充足，使用 int64 可以更好地避免数据溢出的问题。但如果整数数据较小，或者需要节省内存空间，那么使用 int32 可能更合适。

4. 字典方式

```
1 data = {'a': 1, 'b': 2, 'c': 3}
2 s = pd.Series(data)
3 print(s)
```

5. 指定索引

注意：

1. 字典指定的时候 数据为NaN
2. 列表和array指定的时候则需要一至 数据有多少 索引就多少

```
1 data = [1, 2, 3, 4, 5]
2 s = pd.Series(data, index=['a', 'b', 'c', 'd', 'f'])
3 print(s)
```

6. 输出value和索引

```
1 data = [1, 2, 3, 4, 5]
2 s = pd.Series(data, index=['a', 'b', 'c', 'd', 'f'])
3 print(s.value())
4 print(s.index())
```

2.3 series的基本用法

1. isnull 和 notnull 检查缺失值

- isnull: 判断是否为空 空就是True

"isnull"函数返回一个布尔值，用于指示在指定的数据列中哪些值是缺失的（NULL或NaN）。它对于检查整个数据框或单个列中的缺失值非常有用。

```
1 import pandas as pd
2
3 data = {'a': 1, 'b': 2, 'c': 3}
4 s = pd.Series(data, index=['a', 'b', 'c', 'd', 'f'])
5 # 检查整个数据框的缺失值
6 print(s.isnull())
```

- notnull: 判断是否不为空 非空就是True

"notnull"函数返回一个布尔值，用于指示在指定的数据列中哪些值不是缺失的。它可以用来筛选出数据框中不包含缺失值的行或者对缺失值进行计数。

```
1 import pandas as pd
2
3 data = {'a': 1, 'b': 2, 'c': 3}
4 s = pd.Series(data, index=['a', 'b', 'c', 'd', 'f'])
5 # 检查整个数据框的非缺失值
6 print(s.notnull())
```

2. 通过索引获取数据

```

1  import pandas as pd
2
3  data = {'a': 1, 'b': 2, 'c': 3}
4  s = pd.Series(data, index=['a', 'b', 'c', 'd', 'f'])
5
6  # 通过索引位置获取数据
7  print(s[0]) # 获取第一个元素
8  # 选取多个
9  print(s[[2,3]]) # 获取第三个元素
10
11 # 通过索引值获取数据
12 print(s['a']) # 获取索引为'a'的元素
13 # 选取多个
14 print(s[['a','b']]) # 获取索引为'b'的元素
15
16 # 切片取值
17 print(s[1:3]) # 包头不包尾
18 print(s['a':'c']) # 包头包尾
19
20 # bool索引
21 print(s[s>3])
22
23 # 通过索引值获取数据
24 print(s.loc['a']) # 获取索引为'a'的元素
25 print(s.loc[['b', 'c']]) # 获取索引为'b'和'c'的元素

```

3. 索引与数据的对应关系不被运算结构影响

当进行一些运算或操作时，例如合并、过滤、排序等，Pandas 会保持索引与数据之间的对应关系。这在处理数据和分析时非常有用，因为它允许你对数据进行灵活的操作，而不会丢失索引的关联信息。

```

1  import pandas as pd
2
3  data1 = {'A': 1, 'B': 2, 'C': 3}
4  s1 = pd.Series(data1)
5
6  data2 = {'B': 10, 'C': 20, 'D': 30}
7  s2 = pd.Series(data2)
8
9  # 进行加法运算
10 s3 = s1 + s2
11
12 print(s3)
13 print(s3*2)
14 print(s3>3)

```

4. name属性

4.1. 什么是name属性

在 Pandas 中，Series 和 DataFrame 对象都有一个 `name` 属性，它用于给数据对象命名。该属性可以用来标识和标记数据对象，方便后续的认识和操作。

4.2. 对于Series对象，`name` 属性可以通过 `.name` 来访问或设置。它通常用于给 Series 对象本身命名。

```
1 import pandas as pd
2
3 data = [1, 2, 3, 4, 5]
4 s = pd.Series(data, name='MySeries')
5
6 print(s)
7 print(s.name)
8 s.name = 'MySeries' # 对象名字
9 s.index.name='year' # 对象索引名字
10 print(s)
```

6. 获取指定的行数数据

- `head()`

`head()` 是Pandas库中DataFrame和Series对象的一个方法，用于查看对象的前几行数据，默认情况下，它返回前5行。

```
1 # 查看DataFrame的前5行数据
2 print(df.head())
3 print(df.head(3))
```

- `tail(number=5)`

`tail()` 是Pandas库中DataFrame和Series对象的一个方法，用于查看对象的后几行数据，默认情况下，它返回最后5行。

2.4 Series作业

class	
1	90
2	95
3	85
4	78
5	
6	96
7	94
8	
9	80
10	87
11	86
12	83

1. 按照上表创建Series(对象名"score",索引名"class")
2. 查看1-5班的数据
3. 查看哪个班级的成绩还没有录入
4. 获取11班的平均成绩
5. 有一道题的参考答案是错误的, 所以在每位同学现在的成绩上加2分
6. 找出平均成绩在90分以上的班级

```

1  import pandas as pd
2
3  data = [90,95 , 85,78,None,96,None,80,87,86,83 ]
4  s = pd.Series(data,index=range(1,len(data)+1),name="score")
5  s.index.name="class"
6
7
8  print(s.head())
9  print(s.isnull())
10 print(s[11])
11 print(s)
12 print(s+2)
13 print(s>90)
14

```

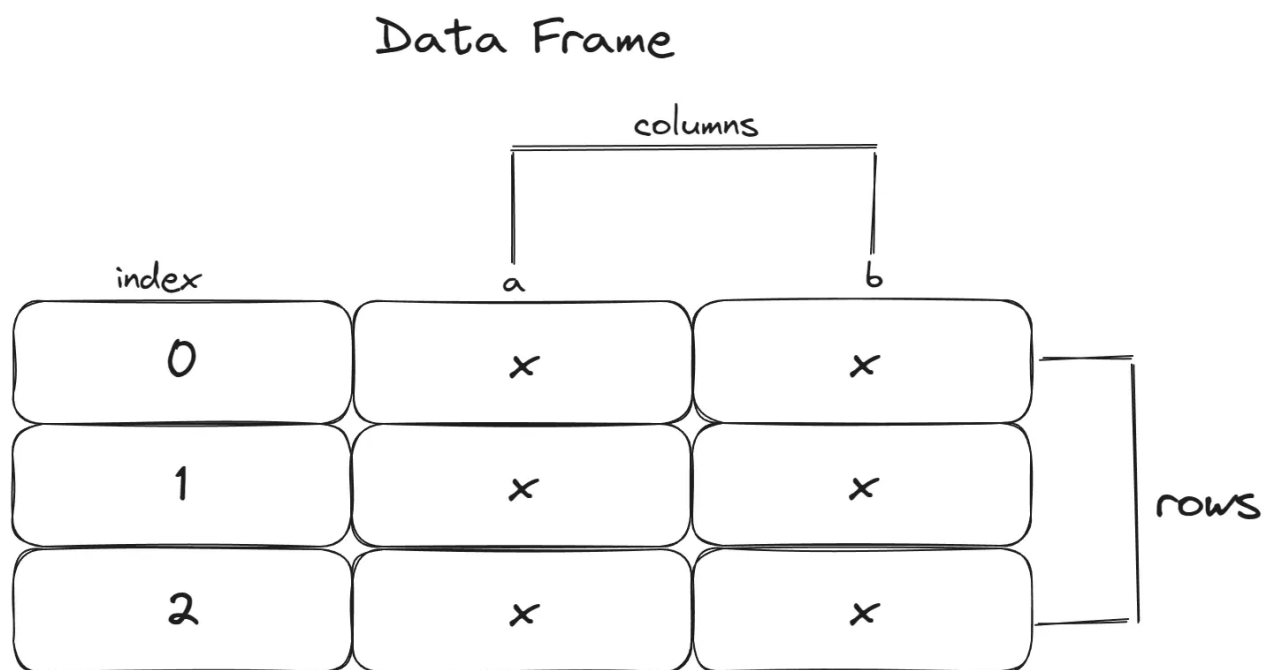
3.DataFrame

3.1 DataFrame的介绍

DataFrame是一个表格型的数据结构，它包含一组有序的列，每列可以是不同类型的值。DataFrame既有行索引也有列索引，它可以被看做是由Series组成的字典（共用同一个索引），数据是以二维结构存

放的。

- 类似多维数组/表格数据(如, excel,,R中的data.frame)
- 每列数据可以是不同的类型
- 索引包括列索引和行索引



3.2 DataFrame创建

1. 字典创建

1.1. 数组、列表或元组构成的字典构造dataframe

```
1 import pandas as pd
2 import numpy as np
3
4 data = {'A': np.arange(9,13), 'B': np.arange(9,13), 'C': np.arange(9,13)}
5 df = pd.DataFrame(data)
6 print(df)
```

```
1 import pandas as pd
2
3 data = {'A': [1, 2, 3, 4], 'B': [5, 6, 7, 8], 'C': [9, 10, 11, 12]}
4 df = pd.DataFrame(data)
5 print(df)
```



```

1 import pandas as pd
2
3 data = {'A': (1, 2, 3, 4), 'B': (5, 6, 7, 8), 'C': (9, 10, 11, 12)}
4 df = pd.DataFrame(data)
5 print(df)

```

index属性查看行索引

```

1 print(df.index)

```

columns属性查看列索引

```

1 print(df.columns)

```

values属性查看值

```

1 print(df.values)

```

指定index和columns

```

1 import pandas as pd
2
3 data = {'A': (1, 2, 3, 4), 'B': (5, 6, 7, 8), 'C': (9, 10, 11, 12)}
4 df = pd.DataFrame(data, index=['A', 'B', 'C', 'D'], columns=['a', 'b', 'c', 'd'])
5 print(df)

```

1.2. Series构成的字典构造dataframe

要使用Series构成的字典来构造一个DataFrame，可以使用pd.DataFrame()函数。DataFrame是Pandas库中用于处理和分析数据的二维数据结构。

```

1 import pandas as pd
2
3 data = {'A': pd.Series([1, 2, 3, 4]), 'B': pd.Series([5, 6, 7, 8]), 'C': pd
4 .Series([9, 10, 11, 12])}
5 df = pd.DataFrame(data)
6 print(df)

```

1.3. 字典构成的字典构造dataframe

```

1 import pandas as pd
2
3 data = {'A': {'a': 1, 'b': 2, 'c': 3}, 'B': {'a': 4, 'b': 5, 'c': 6}, 'C':
4        {'a': 7, 'b': 8, 'c': 9}}
5 df = pd.DataFrame(data)
6 print(df)

```

在创建DataFrame时，外层字典的键将成为DataFrame的列标签，而内层字典的键将成为DataFrame的索引。

2. 列表创建

2.1. 2D ndarray构造dataframe

```

1 import pandas as pd
2 import numpy as np
3
4 data = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
5 #data = np.arange(12).reshape(4,3)
6 # df = pd.DataFrame(data, index=['A', 'B', 'C'], columns=['X', 'Y', 'Z'])
7 df = pd.DataFrame(data)
8
9 print(df)

```

2.2. 字典构成的列表构造dataframe

```

1 import pandas as pd
2
3 # 字典构成的列表
4 data = [
5     {"Name": "John", "Age": 25, "City": "New York"},
6     {"Name": "Emily", "Age": 28, "City": "London"},
7     {"Name": "James", "Age": 22, "City": "Paris"}
8 ]
9
10 # 将列表转换为 DataFrame
11 df = pd.DataFrame(data)
12
13 # 打印 DataFrame
14 print(df)

```

2.3. Series构成的列表构造dataframe

```

1  import pandas as pd
2
3  # Series 构成的列表
4  series_list = [
5      pd.Series([1, 2, 3]),
6      pd.Series([4, 5, 6]),
7      pd.Series([7, 8])
8  ]
9
10 # 将列表转换为 DataFrame
11 df = pd.DataFrame(series_list)
12
13 # 打印 DataFrame
14 print(df)

```

3.3 DataFrame基本用法

```

1  pd5 = pd.DataFrame(np.arange(9).reshape(3,3), index=['a', 'c', 'b'], columns=[
    'A', 'B', 'C'])
2  print(pd5)

```

1. T转置：行与列进行转换

```

1  import pandas as pd
2  import numpy as np
3
4  pd5 = pd.DataFrame(np.arange(9).reshape(3,3), index=['a', 'c', 'b'], columns=[
    'A', 'B', 'C'])
5  pd5_transposed = pd5.T
6
7  print(pd5_transposed)

```

2. 通过列表索引获取列数据(Series类型)

```

1  print(pd5['A'])
2
3  column_index = ['A', 'C']
4  columns_data = pd5[column_index]
5
6  print(columns_data)

```

3. 增加列数据

```

1 # 增加新的列数据
2 pd5['D'] = [9, 10, 11]
3
4 # 打印增加列数据后的DataFrame
5 print(pd5)

```

4. 删除列

```

1 import pandas as pd
2 import numpy as np
3
4 pd5 = pd.DataFrame(np.arange(9).reshape(3,3), index=['a','c','b'], columns
5 =['A','B','C'])
6 pd5_transposed = pd5.T
7
8 # 删除指定的列
9 pd5_transposed = pd5_transposed.drop(['B'], axis=1)
10
11 # 打印删除列后的DataFrame
12 print(pd5_transposed)

```

3.4 DataFrame作业

1	姓名	语文	数学	英语	体育
2	张三	89	59	84	0
3	李四	78	83	97	0
4	王五	79	85	88	0
5	小明	89	92	83	0
6	小红	90	67	67	0
7	小刚	87	81	73	0
8	小亮	83	77	71	0

1. 根据上图创建一个DataFrame（列表和字典）
2. 对于创建的DataFrame进行转置
3. 由于体育老师总是生病，成绩没有录入，所以删除该列数据
4. 学校决定添加"综合"这个列，学生的这列的成绩分别是(97,87,78,76,84,88,91)

```
1 data={
2     1: {"姓名": "龙仔", "语文": random.randint(0,100), "数学": random.randint(0,100),
3         "英语": random.randint(0,100), "体育": 0},
4     2: {"姓名": "麻辣龙虾", "语文": random.randint(0,100), "数学": random.randint(0,100),
5         "英语": random.randint(0,100), "体育": 0},
6     3: {"姓名": "小炒虾", "语文": random.randint(0,100), "数学": random.randint(0,100),
7         "英语": random.randint(0,100), "体育": 0},
8     4: {"姓名": "油焖大虾", "语文": random.randint(0,100), "数学": random.randint(0,100),
9         "英语": random.randint(0,100), "体育": 0},
10    5: {"姓名": "帝王虾", "语文": random.randint(0,100), "数学": random.randint(0,100),
11        "英语": random.randint(0,100), "体育": 0},
12 }
13 pd1 = pd.DataFrame(data)
```

3.pandas的基本操作

3.1 索引操作

索引对象index

1. Series和DataFrame中的索引都是index对象

示例代码:

```

1  import numpy as np
2  import pandas as pd
3
4  ps = pd.Series(range(5), index=["a", "b", "c", "d", "e"])
5  print(ps)
6
7  pd1 = pd.DataFrame(np.arange(9).reshape(3,3), index=["a", "b", "c"], columns=[
    "A", "B", "C"])
8  print(pd1)
9
10 print(type(ser_obj.index))
11 print(type(df_obj2.index))
12 print(df_obj2.index)

```

2. 索引对象不可变,保证了数据的安全

```

1  ps['a']=2
2  print(ps)
3
4  ps.index['a'] = 2
5  print(ps)
6
7  pd1.index["a"]=3
8  print(pd1)

```

3. 常见的index的种类

- a. index , 索引
- b. int64index, 整数索引
- c. MultiIndex, 层级索引
- d. DatetimeIndex 时间戳索引

3.2 重新索引

重新索引是指在 Pandas 中对现有的数据结构（如 Series、DataFrame）进行重新排序、添加、删除索引的操作。

reindex() 方法来实现。

```

1  import numpy as np
2  import pandas as pd
3
4  ps = pd.Series(range(5), index=["a", "b", "c", "d", "e"])
5  print(ps)
6  # reindex 创建一个新的对象，而不改变原始数据
7  ps2 = ps.reindex(["a", "b", "c", "d", "e"])
8  print(ps2)
9
10 pd1 = pd.DataFrame(np.arange(9).reshape(3,3), index=["a", "b", "c"], columns=[
    "A", "B", "C"])
11 print(pd1)
12
13 # 行索引重建
14 pd2 = pd1.reindex(["a", "b", "c", "d"])
15 print(pd2)
16
17 # 列索引重置
18 pd3 = pd1.reindex(columns=["C", "B", "A"])
19 print(pd3)

```

3.3 增删改查操作

3.3.1 增加操作

1. Series

a. 增加

```

1  import numpy as np
2  import pandas as pd
3
4  ps = pd.Series(range(5), index=["a", "b", "c", "d", "e"])
5  print(ps)
6  ps['f'] = 9
7  print(ps) # 是在原有的基础上增加
8
9  # 怎么在不修改原有数据上进行添加呢?
10 s1 = pd.Series({'g': 999})
11 ps2 = ps.append(s1)
12 print(ps2)
13 print(ps)

```

b. 删除

```

1  import numpy as np
2  import pandas as pd
3
4  ps = pd.Series(range(5), index=["a", "b", "c", "d", "e"])
5  print(ps)
6  # del
7  del ps['b'] # 这种方式不推荐 在原有对象上修改
8  print(ps)
9
10 # drop 删除轴上数据
11 ps1 = ps.drop('a') # 不会在原有的对象上进行修改 是创建一个新的对象 并删除 返回
12 print(ps)
13 print(ps1)
14
15 # 删除多行
16 ps2 = ps.drop(["a", "c", "d"])
17 print(ps2)

```

c. 修改

```

1  import numpy as np
2  import pandas as pd
3
4  ps = pd.Series(range(5), index=["a", "b", "c", "d", "e"])
5  print(ps)
6
7  ps['a'] = 999
8  print(ps)
9
10 ps[0] = 888
11 print(ps)
12

```

d. 查询


```
1 import numpy as np
2 import pandas as pd
3
4 ps = pd.Series(range(5), index=["a", "b", "c", "d", "e"])
5 print(ps)
6 # 1, 行索引
7 print(ps['a'])
8
9
10 # 2. 多数据查询
11 # 位置切片索引
12 print(ps[1:4])
13 # 标签切片
14 print(ps["a":"e"])
15
16 # 不连续索引
17 print(ps1[['b', 'e']])
18
19 #位置索引
20 print(ps1[[0,2,3]])
21
22 # 布尔索引
23 print(ps1[ps1>2])
```

2. DataFrame

a. 增加

```

1  import numpy as np
2  import pandas as pd
3
4  pd1 = pd.DataFrame(np.arange(9).reshape(3,3),index=["a","b","c"],columns=[
    "A","B","C"])
5  print(pd1)
6
7  # 增加列
8  pd1[4] = 9 # 这里所表示的 增加是一列 默认的 数据全是9
9  print(pd1)
10 # 不想让数据一模一样
11 pd1[4] = [10,20,30] # 增加是一列 数据10 20 30
12 print(pd1)
13
14 # 插入操作 insert(插入的位置,插入的索引,插入的数据)
15 pd1.insert(0,"E",[9,99,999])
16 print(pd1)
17
18 # 增加行
19 # 标签索引 loc
20 pd1.loc["d"] = [1,2,3,4]
21 print(pd1)
22
23 row = {"E":6,"A":6,"B":6,"C":6,4:6}
24 pd2 = pd1._append(row)
25 print(pd2) # 报错 要忽略
26
27 pd2 = pd1._append(row,ignore_index=True)
28 print(pd2)

```

b. 删除

```

1  import numpy as np
2  import pandas as pd
3
4  pd1 = pd.DataFrame(np.arange(9).reshape(3,3),index=["a","b","c"],columns=[
    "A","B","C"])
5  print(pd1)
6
7  del pd1["E"] # 删除的是列
8  print(pd1)
9
10 # 删除行
11 pd2 = pd1.drop('a') # 这里删除的是行
12 print(pd2)
13 # 删除多行
14 pd3 = pd1.drop('a','b')
15 print(pd3)
16
17 # 指定删除列 drop(索引,轴=0,)
18 pd4 = pd1.drop('A',axis=1) # 1是列 0是行 默认为0
19 print(pd4)
20
21 pd5 = pd1.drop('A',axis='columns') # 等同于上面的效果
22 print(pd5)
23
24 # drop 的 inplace属性 是否在原对象删除 默认为False
25 pd1.drop("a",inplace=True)
26 print(pd1)

```

c. 修改

```
1 import numpy as np
2 import pandas as pd
3
4 pd1 = pd.DataFrame(np.arange(9).reshape(3,3), index=["a","b","c"], columns=[
    "A","B","C"])
5 print(pd1)
6
7 # 修改一列
8 pd1['A'] = 9
9 print(pd1)
10
11 # 多值
12 pd1['A'] = [9,10,11]
13 print(pd1)
14
15 # 不太常见 对象.列 方式
16 pd1.A = 6
17 print(pd1)
18
19
20 #loc 标签索引 行索引修改
21 pd1.loc['a'] = 777
22 print(pd1)
23
24 # 指定位置实现
25 pd1.loc['a','A'] = 1000
26 print(pd1)
27
28
```

d. 查询

```

1  import numpy as np
2  import pandas as pd
3
4  pd1 = pd.DataFrame(np.arange(9).reshape(3,3),index=["a","b","c"],columns=[
    "A","B","C"])
5  print(pd1)
6
7  # 列索引
8  print(pd1['A'])
9  # 取多列
10 print(pd1[['A','C']])
11
12 # 选取一个值
13 print(pd1['A']['a'])
14
15 # 切片
16 print(pd1[:2]) # 获取行
17
18 # 怎么切列呢?

```

3.4 高级索引

1. loc标签索引

loc是基于标签名的索引 自定义的索引名称

```

1  import numpy as np
2  import pandas as pd
3
4  ps = pd.Series(range(5),index=["a","b","c","d","e"])
5  print(ps)
6  #Series
7  print(ps['a':'c'])
8  print(ps.loc['a':'c'])
9
10
11 pd1 = pd.DataFrame(np.arange(9).reshape(3,3),index=["a","b","c"],columns=[
    "A","B","C"])
12 print(pd1)
13 # 标签切片
14 pd1.loc['a':'b',"A"]# 第一个参数索引行 第二个是列
15
16 pd1.loc['a':'b',"A":"C"]

```

2. iloc位置索引

```
1 import numpy as np
2 import pandas as pd
3
4 ps = pd.Series(range(5), index=["a", "b", "c", "d", "e"])
5 print(ps)
6 print(ps[1:3])
7 print(ps.iloc[1:3])
8
9
10 pd1 = pd.DataFrame(np.arange(9).reshape(3,3), index=["a", "b", "c"], columns=[
    "A", "B", "C"])
11 print(pd1)
12 print(pd1.iloc[0:2,0])
13 print(pd1.iloc[0:2,0:3])
14
```

3.5 练习

📎 student.xlsx

1. 观察结构，调整列索引顺序为(id,学生id,学生姓名,年龄,性别,语文成绩,数学成绩,英语成绩,学生是否毕业,学生是否休学)
2. 增加一列关于学生状态(可使用1代码 在 0代表不在)
3. 删除id这一列
4. 查找出学生语文成绩大于90分的并输出的时候只有学生id-语文成绩

准备:读取xlsx

```
1 students = pd.read_Excel(r'student.xlsx')
```

```

1  import random
2
3  import pandas as pd
4  students = pd.read_excel(r'student.xlsx')
5  print(students)
6  '''
7  1. 观察结构, 调整列索引顺序为(id, 学生id, 学生姓名, 年龄, 性别, 语文成绩, 数学成绩, 英语成绩,
   学生是否毕业, 学生是否休学)
8  2. 增加一列关于学生状态(可使用1代码 在 0代表不在)
9  3. 删除id这一列
10 4. 查找出学生语文成绩大于90分的
11 '''
12 students = students.reindex(columns=["id", "学生id", "学生姓名", "年龄", "性别",
   "语文成绩", "数学成绩", "英语成绩", "学生是否毕业", "学生是否休学"])
13 print(students)
14 # apply() 方法将会遍历 学生是否休学 列的每个值, 并将条件逻辑应用到每个值上。
15 # 参数 x 代表的是每个元素值, 也就是 学生是否休学 列中的每个值。
16 students["学生状态"] = students['学生是否休学'].apply(lambda x: 1 if x ==
   "是" else 0)
17 print(students)
18 # 删除id这一列
19 students = students.drop('id', axis=1) # 1是列 0是行 默认为0
20 print(students)
21 # 查找出学生语文成绩大于90分的并输出的时候只有学生id-语文成绩
22 students = students[students['语文成绩'] > 90]
23 print(students.loc[:, "学生id": "语文成绩"])

```

4. Pandas对齐运算

4.1 算术运算和数据对齐

```

1  import numpy as np
2  import pandas as pd
3  #series
4  s1 = pd.Series(np.arange(4),index=['a','b','c','d'])
5  s2 = pd.Series(np.arange(5),index=['a','c','e','f','g'])
6  print(s1)
7  print(s2)
8  print(s1+s2)
9
10
11 #DataFrame
12 df = pd.DataFrame(np.arange(12).reshape(4,3),index=['a','b','c','d'],columns=list('ABC'))
13 df2 = pd.DataFrame(np.arange(9).reshape(3,3),index=['a','d','g'],columns=list('ABD'))
14 print(df)
15 print(df2)
16 print(df+df2)

```

4.2 使用填充值的算术方法

方法	描述
add, radd	加法 (+)
sub, rsub	减法 (-)
div, rdiv	除法 (/)
floordiv, rfloordiv	整除 (//)
mul, rmull	乘法 (*)
pow, rpow	幂次方 (**)

字母r开头 会翻转参数

```

1  s1.add(s2,fill_value=0)
2
3  df1.add(df2,fill_value=0)
4
5  df1.reindex(columns=df2.columns,fill_value=9)

```


4.3 DataFrame和Series混合运算

```
1 import pandas as pd
2 import numpy as np
3 df1= pd.DataFrame(np.arange(12).reshape(4,3),index=['a','b','c','d'],columns=list('ABC'))
4 df2 = pd.DataFrame(np.arange(9).reshape(3,3),index=['a','d','g'],columns=list('ABD'))
5 print(df1)
6
7 s3 = df1.iloc[0]
8 print(s3)
9 print(df1-s3)
10
11 # 列
12 s4 = df1["A"]
13 print(s4)
14 df3 = df1.sub(s4,axis=0)
15 print(df3)
```

5.Pandas的函数应用

5.1 apply和applymap

```
1 import pandas as pd
2 import numpy as np
3 df1= pd.DataFrame(np.random.randn(5,4))
4 print(df1)
5
6 # 可以直接使用numpy的函数
7 print(np.abs(df1))
8
9 # 通过apply将函数应用到列或者行
10 f = lambda x:x.max()
11 print(df.apply(f)) # 列 注意轴的方向 默认是 axis 0 列
12
13 print(df.apply(f,axis=1)) # 行
14
15 # 通过applymap 将函数应用到每个函数
16 f2 = lambda x: '%.2f'%x
17 df.applymap(f2)
```

5.2 排序

1. 索引排序

```
1 import pandas as pd
2 import numpy as np
3
4 s1 = pd.Series(np.arange(4), index=list('dbca'))
5 print(s1)
6
7 print(s1.sort_index()) # 默认升序 按照索引
8
9 print(s1.sort_index(ascending=False)) # 降序
10
11 pd1 = pd.DataFrame(np.arange(12).reshape(4,3), index=list('bdca'), columns=list('BCA'))
12 print(pd1)
13
14 # 按照行排序
15 print(pd1.sort_index())
16
17 # 按照列排序
18 print(pd1.sort_index(axis=1))
```

2. 按值排序

```
1 #Series
2 print(s1.sort_values()) # 根据值的大小进行排序 当值有缺失值，会默认排在最
3 print(s1.sort_values(ascending=False))
4
5 #DataFrame
6 pd1.sort_values(by='A') #指定A列排序
7 pd1.sort_values(by='a', axis=1) #按照行
8
9 #按照多列进行排序 首先按照列'A'进行排序，如果有相同的值，则按照列'B'进行排序。
10 pd1.sort_values(by=['A', 'B'])
```

5.3 唯一值和成员属性

```

1  import pandas as pd
2  import numpy as np
3  s1 = pd.Series([2,6,8,9,8,3,6])
4  print(s1)
5  s2 = s1.unique() #返回一个唯一值数组
6  print(s2)
7
8
9  s1.index=['a','a','a','c','c','c','c']
10 print(s1)
11 print(s1.is_unique) # 如果对象中的值是唯一的，则返回布尔值。
12
13 s1 = pd.Series([2,6,8,9,8,3,6])
14 s1.value_counts() #返回一个series
15
16
17 # 成员属性 什么是成员属性 就是判定值是否存在
18 s1.isin([8]) #判断8是否存在s1
19
20
21 data = pd.DataFrame({'a':[3,7,8,0], 'b':[1,-1,4,7], 'c':[0,6,-3,2]})
22 print(data)
23
24 data.isin([2,4])

```

5.4 处理缺失数据

```

1 pdf3 = pd.DataFrame([np.random.randn(3), [1.,2.,np.nan],
2                      [np.nan,4,np.nana], [1,2.,3.]
3                      ])
4 # 1.判断是否存在是否存在缺失值 isnull()
5 print(df3.isnull())
6 # 2. 丢弃缺失数据 dropna()
7 print(df3.dropna()) # 默认丢弃行
8 print(df3.dropna(axis=1)) # 列
9
10 # 3.填充缺失数据
11 df3.fillna(-1.)

```

6.Pandas-层级索引

1. 层级索引

```

1  import numpy as np
2  import pandas as pd
3
4  s1 = pd.Series(np.random.randn(12), index=[['a', 'a', 'a', 'b', 'b', 'b', 'c', 'c',
        'c', 'd', 'd', 'd']])
5  print(s1)
6
7  s1 = pd.Series(np.random.randn(12),
8  index=[['a', 'a', 'a', 'b', 'b', 'b', 'c', 'c', 'c', 'd', 'd', 'd'],
9  , [0, 1, 2, 0, 1, 2, 0, 1, 2, 0, 1, 2]])
10
11 frame = pd.DataFrame(np.arange(12).reshape((4, 3)),
12 index=[['a', 'a', 'b', 'b'], [1, 2, 1, 2]],
13 columns=[['Ohio', 'Ohio', 'Colorado'],
14 ['Green', 'Red', 'Green']])
15
16 print(s1)
17
18 print(type(s1.index))
19
20 #层级 第一个层级 第二个位置
21 print(s1.index)
22

```

2. 选取

```

1  import numpy as np
2  import pandas as pd
3  s1 = pd.Series(np.random.randn(12),
4  index=[['a', 'a', 'a', 'b', 'b', 'b', 'c', 'c', 'c', 'd', 'd', 'd'],
5  , [0, 1, 2, 0, 1, 2, 0, 1, 2, 0, 1, 2]])
6  # 外层
7  print(s1['b'])
8
9  # 内存
10 print(s1[:, 2])
11
12 # 具体数据
13 print(s1['a', 0])

```

3. 交换

```

1  print("-----")
2
3  # 交接外层 内存索引
4  print(s1.swaplevel())
5  #设置index的levels的名称, key1 与 key2分别对应不同的层
6  s1.index.names = ['key1', 'key2']
7
8
9  #设置columns的levels的名称
10 # s1.columns.names = ['state','color']
11
12 # 交换外层 内存索引 并排序 先外排序在内排序 升序 在交换 根据索引排序 并不是值排序
13 print(s1.sort_index(level='key2',ascending=False))
14 print("-0-----")
15 print(s1.sort_values(ascending=False))
16 # 交换并排序分层
17 # print(s1.swaplevel().sortlevel())
18
19 data = {
20     'a': [12,92,93,0] ,
21     'b': [62,97,99,2] ,
22     'c': [93,45,87,64]
23 }
24 pd1 = pd.DataFrame(
25     data
26 )
27 pd1.index = [['b','c','a','d'],[2,3,6,1]]
28 pd1.columns= [['B','C','A'],[3,1,2]]
29 pd1.index.names = ['key1','key2']
30 pd1.columns.names = ['col1','col2']
31 print(pd1)
32 print(pd1['B'])
33 print(pd1.loc['a','B'])
34
35 # 排序
36 print(pd1.sort_values(by=[("A",2),("B",3)]))
37

```

7.Pandas统计计算和描述

```

1 import numpy as np
2 import pandas as pd
3
4 arr1 = np.random.rand(4,3)
5 pd1 = pd.DataFrame(arr1,columns=list('ABC'),index=list('abcd'))
6 f = lambda x: '%.2f'%s
7 pd2 = pd1.applymap(f).astype(float)
8 print(pd2)

```

常用的统计计数

sum. mean, max, min...

方法	说明
count	非NA值的数量
describe	针对Series或各DataFrame列计算汇总统计
min,max	计算最小值和最大值
idxmin、idxmax	计算能够获取到最小值和最大值的索引值
quantile	计算样本的分位数（0到1）
sum	值的总和
mean	值的平均数
median	值的算术中位数(50%分位数)
var	样本值的方差
std	样本值的标准差
cumsum	样本值的累计和

```

1  import pandas as pd
2  import numpy as np
3  pd1 = pd.DataFrame(
4      [[1.4,np.nan],[7.1,np.nan],[4.6,7.8],[np.nan,np.nan],[0.75,0.3]]
5      ,index=['a','b','c','d','e'],columns=['A','B']
6  )
7  print(pd1)
8
9  # 1. count(): 计算非缺失值的数量。
10 print(df.count())
11 # 2. describe
12 '''
13 describe() 函数会计算每个数值列的统计特征,
14 包括计数、平均值、标准差、最小值、25% 四分位数、中位数、75% 四分位数和最大值。
15 '''
16 print(df.describe()) # 对于 Series 对象 但没有四分位数
17
18 # 3. min(): 计算数值列的最小值。
19 print(df.min())
20 print(df[0].min())
21
22 # 4. max(): 计算数值列的最大值。
23 print(df.max())
24 print(df[0].max())
25
26 # 5. 计算能够获取到最小值和最大值的索引值
27 print(df.idxmin())
28
29 # 6. 可以设置值 0.5 0.25
30 print(df.quantile(0.5))
31 # 计算其他分位数, 比如 25% 和 75%
32 print(df.quantile([0.25, 0.75]))
33
34
35 # 7. sum(): 计算数值列的总和。
36 print(df.sum())
37 print("-----")
38 print(df.sum(axis=1,skipna=True))
39 print(df[0].sum())
40
41 # 8. mean(): 计算数值列的平均值。
42 print(df.mean())
43 print(df[0].mean())
44
45 # 9. median(): 计算数值列的中位数。
46 print(df.median())
47 print(df[0].median())

```

```

48
49
50 # 10. var(): 计算数值列的方差。
51 print(df.var())
52 print(df[0].var())
53
54 # 11. std(): 计算数值列的标准差。
55 print(df.std())
56 print(df[0].std())
57
58 # 12. cumsum
59 # 创建一个 Series 对象
60 data = pd.Series([1, 2, 3, 4, 5])
61
62 # 计算累积和
63 cumulative_sum = data.cumsum()
64 print(cumulative_sum)
65
66

```

8.Pandas入门总结



9.数据加载，存储与文件格式

前面的课程中都是我们自己定义的数据那么接下我们将要学习 如果从外部获取数据 也就是数据加载 对数据进行进一步的数据分析和处理。也就是pandas的输出与输入 当然我们是输入与输出分为几大类

9.1 读取文件格式文件 (熟悉)

方法	说明
pd.read_csv()(重点)	从文件、URL、文件型对象中加载带分隔符的数据，默认分隔符为逗号
pd.read_table()	从文件、URL、文件型对象中加载带分隔符的数据，默认分隔符为制表符 ("\\t")
pd.read_fwf()	读取定宽列格式数据(也就是说，没有分隔符)
pd.read_clipboard()	读取剪贴板中的数据，可以看做read_table的剪贴板版。再将网页转换为表格时很有用
pd.read_excel()(重点)	从Excel XLS或XLSX file读取表格放据
pd.read_hdf()	读取pandas写的HDF5文件
pd.read_html()(重点)	读取HTML文档中的所有表格
pd.read_json()(重点)	读取JSON (JavaScript Object Notation)字符串中的数据
pd.read_Parquet()	读取 Parquet 文件的函数。它可以将 Parquet 文件加载为 Pandas DataFrame
pd.read_pickle()	读取Python pickle格式中存储的任意对象
pd.read_sas()	读取存储于SAS系统自定义存储格式的SAS数据集
pd.read_sql()(重点)	(使用SQLAlchemy)读取SQL查询结果为pandas的DataFrame
pd.read_stata()	读取stata文件格式的数据集

参数	说明
path	表示文件系统位置、URL,文件型对象的字符串
sep或delimiter	用于对行中各字段进行拆分的字符序列或正则表达式
header	用作列名的行号。默认为0（第一行），如果没有header行就应该设置为None
index_col	用作行索引的列编号或列名。可以是单个名称/数字或由多个名称/数字组成的列表（层次化索引）

names	用于结果的列名列表，结合header=None
skiprows	需要忽略的行数（从文件开始处算起），或需要跳过的行号列表（从0开始）
na_values	一组用于替换NA的值
comment	用于将注释信息从行尾拆分出去的字符（一个或多个）
parse_dates	尝试将数据解析为日期，默认为False。如果为True,则尝试解析所有列。此外，还可以指定需要解析的一组列号或列名。如果列表的元素为列表或元组，就会将多个列组合到一起再进行日期解析工作（例如，日期/时间分别位于两个列中）
keep_date_col	如果连接多列解析日期，则保持参与连接的列。默认为False.
converters	由列号/列名跟函数之间的映射关系组成的字典。例如，{"foo":f}会对foo列的所有值应用函数f
dayfirst	当解析有歧义的时间日期时，将其看做国际格式（例如，7/6/2012—June 7,2012),默认为False
date_parser	用于解析日期的函数
nrows	需要读取的行数（从文件开始处算起）
iterator	返回一个TextParser以便逐块读取文件
chunksize	文件块的大小（用于迭代）
skip_footer	需要忽略的行数（从文件末尾处算起）

9.1.1 读取

9.1.1 read_csv()

从文件、URL、文件型对象中加载带分隔符的数据，默认分隔符为逗号

```
1 import pandas as pd
2 pd1 = pd.read_csv('文件的路径')
3 print(pd1)
```

注意：在读取没有设置标题的数据的时候 他会把我们的第一行当作是标题

但是可以通过参数 header 的设置为默认的列名

```
1 import pandas as pd
2 pd1 = pd.read_csv('文件的路径', header=None) #使用默认的列名
3 print(pd1)
```

也可以通过names设置自定义的列名

```
1 import pandas as pd
2 pd1 = pd.read_csv('文件的路径', names=['A', 'B', 'C', 'D', 'message'])
3 print(pd1)
```

在一些特殊情况下 需要将某一列变成行索引 可以通过参数 index_col进行设置

```
1 import pandas as pd
2 pd1 = pd.read_csv('文件的路径', names=['A', 'B', 'C', 'D', 'message'], index_col=
  '某列')
3
4 print(pd1)
```

设置层级索引

```
1 import pandas as pd
2 pd1 = pd.read_csv('文件的路径', names=['A', 'B', 'C', 'D', 'message'], index_col=[
  '列1', '列2'])
3
4 print(pd1)
```

9.1.2 read_table()

从文件、URL、文件型对象中加载带分隔符的数据，默认分隔符为制表符 ("\t")

```

1 list(open('文件路径'))
2
3 import pandas as pd
4 pd1 = pd.read_table('文件的路径.txt')
5 print(pd1)

```

1.注意：读取出来的数据和csv读取的格式不一样 因为 csv使用逗号所以会忽略 而table是\t 可以通过参数的设置进行

```

1 import pandas as pd
2 pd1 = pd.read_table('文件的路径',sep=',')
3 print(pd1)

```

读取：txt文本

```

1 import pandas as pd
2 pd1 = pd.read_table('txt',sep='\s+')
3 print(pd1)

```

9.1.3 read_fwf()

用于将固定宽度格式（FWF）的文件读取到 pandas 的 DataFrame 中。FWF 文件是以固定宽度的列组织数据的文本文件。

```

1 import pandas as pd
2
3 df = pd.read_fwf('data.fwf', colspecs=[(0, 5), (6, 10), (11, 15)], names=[
    '列1', '列2', '列3'])
4 print(df)

```

9.1.4 read_clipboard()

它可以从剪贴板读取数据，并将其加载到 pandas 的 DataFrame 中

使用 `pd.read_clipboard()` 的步骤如下：

1. 复制要读取的数据到剪贴板（可以是表格形式的数据）。
2. 运行 `pd.read_clipboard()` 代码。

```
1 import pandas as pd
2
3 df = pd.read_clipboard()
4
5 print(df)
```

请注意，`pd.read_clipboard()` 函数依赖于剪贴板中的数据的格式正确性。确保复制的数据在表格中是对齐的，并且在运行代码之前没有对剪贴板进行其他操作。

9.1.5 read_excel()

用于从 Excel 文件中读取数据并将其加载到 pandas 的 DataFrame 中。

```
1 import pandas as pd
2
3 df = pd.read_excel('文件路径/文件名.xlsx')
4
5 print(df)
```

9.1.7 read_html()

```
1 import pandas as pd
2 pd1= pd.read_html("https://www.dxsbb.com/news/131783.html",encoding="gbk")
3
4
5 print(pd1)
```

读取成功后，HTML 文件中的表格数据将加载到一个列表中，其中每个元素都是一个 DataFrame 对象。存储在列表中的 DataFrame 对象可以根据需要进行进一步的访问和处理。

请注意，`pd.read_html()` 并不能读取 HTML 文档中的所有内容，它只能读取包含表格数据的 HTML 表格标签。此外，如果 HTML 文件中有多个表格，`pd.read_html()` 会将每个表格都读取为一个 DataFrame 对象，并存储在列表中返回。

如果您的 HTML 文件中包含多个表格，并且您只想读取其中的某个特定表格，您可以通过指定 `match` 参数来筛选要读取的表格。例如：

```
1 dfs = pd.read_html('文件路径/文件名.html', match='表格标识符')
```

您需要将 '表格标识符' 替换为您想要读取的表格的标识符或 CSS 选择器。这样，`pd.read_html()` 函数将只返回匹配标识符的表格数据。

注意:

请注意, 对于一些复杂的 HTML 文件, `pd.read_html()` 可能无法正确读取表格数据。在这种情况下, 您可能需要使用其他库或方法来解析和提取您需要的数据。

9.1.8 Json

1. 什么是json

JSON (JavaScript Object Notation) 是一种常用的数据交换格式, 它以易于人类阅读和编写的方式表示数据。它基于 JavaScript 的一个子集, 但现在被广泛地用于不同编程语言间的数据传输和存储。

JSON 使用键值对的方式来组织数据, 数据可以是字符串、数字、布尔值、数组、对象等

JSON 数据使用大括号 `{}` 表示对象, 对象中的每个键值对使用冒号 `:` 分隔, 键值对之间使用逗号 `,` 分隔。例如

```
1 import pandas as pd
2 import json
3 obj = '{"queryExt": "小姐姐","listNum": 1974}'
4 # 将json数据转换成python对象
5 res = json.loads(obj)
6 print(res)
7 print(type(res))
8
9 # python对象转换成json数据
10 res1 = json.dumps(res)
11 print(res1)
12 print(type(res1))
13 print(res['queryExt'])
14 pd1 = pd.DataFrame(res, columns=['queryExt', 'listNum'], index=[1,0])
15 print(pd1)
```

9.1.9 分块读取大文件

分块读取大文件的主要目的是为了优化内存的使用和提高读取的效率。当处理大文件时, 将整个文件一次性加载到内存中可能会导致内存不足的问题, 特别是在处理非常大的文件时。此外, 直接一次性读取整个文件可能会导致读取时间过长, 影响程序的响应性能。

通过分块读取大文件, 可以将文件分割成较小的块, 逐块读取并处理。这种方式有以下好处:

1. 节省内存: 只需要将文件的一部分加载到内存中, 而不是加载整个文件, 可以减少内存的消耗。
2. 提高读取速度: 按块读取文件可以避免一次性读取整个文件的时间开销。因为文件操作往往是相对较慢的, 逐块读取可以将文件的读取时间分摊到多个较小的操作中, 从而提高读取速度。
3. 更好的响应性能: 如果文件读取过程涉及与用户交互或其他任务并发执行, 分块读取可以使程序更加响

应，减少等待时间。

分块读取大文件的具体实现可以使用文件流（File Stream）或者按行读取文件，根据具体需求和文件格式选择合适的方式。

```
1  # coding: utf-8
2  '''
3  @FileName: 11.分块读取大文件.py
4  @Modify : 2023/10/16 17:12
5  @Description : None
6  '''
7  import pandas as pd
8
9  agg1 = pd.read_csv("D:\Pandas\day09-数据加载_存储_文件格式\二手车之家信息 - 副本.csv") # 任务管理器
10 agg1.head(10) # 这种是不是分块读取 : 不是
11 # 那怎么分块读取呢?
12 agg1 = pd.read_csv("D:\Pandas\day09-数据加载_存储_文件格式\二手车之家信息 - 副本.csv", chunksize=10) # 设置10行
13 print(agg1, type(agg1)) # 返回的是一个可迭代对象
14 print(agg1.get_chunk()) # 读取下一行
15 print(type(agg1))
16
17 print(agg1.get_chunk(5)) # 进行设置 就获取5个数据
18 print(agg1.get_chunk()) # 不进行设置还是10行数据
19
20 #iterator=True 意味着启用迭代器模式，这将返回一个迭代器对象而不是完整的DataFrame。
21 #在迭代器模式下，数据将按块读取，而不是一次性将整个DataFrame加载到内存中。
22 #每次调用 __next__() 方法时，将返回一个包含指定行数的DataFrame块。
23 #当到达数据的末尾时，__next__() 方法将引发 StopIteration 异常。
24 #使用 iterator=True 可以用于处理大型数据文件，以避免内存不足的问题。
25 #通过按块读取数据，可以减少内存使用并提高效率。
```

9.1.2 写入

```
1  import pandas as pd
2  pd1 = pd.read_csv('csv.文件的路径', names=['A', 'B', 'C', 'D', 'message'], index_col=['列1', '列2'])
3
4  print(pd1)
5
6  pd2 = pd1.to_csv('写入的文件的名字.格式')
7  print(pd2)
```

9.2 二进制数据格式(了解)

```
1 frame = pd.read_csv('.csv文件')
2 print(frame)
3 # 实现数据的高效二进制格式存储的最简单的方法, 使用python内置的pickle序列化
4 # pandas对象都有一个用于将数据以pickle格式保存到磁盘上的to_pickle方法
5 frame.to_pickle('frame_pickle')
6 pd.read_pickle('frame_pickle')
```

HDF5格式

HDF5 (Hierarchical Data Format 5) 是一种用于存储和管理大规模科学数据集的文件格式和库。它是一种开放的、高效的数据模型, 旨在支持处理和分析各种类型和规模的科学数据。

HDF5以层次结构的方式组织数据, 可以存储多维数组、表格、图像和其他复杂的数据对象。它还支持数据的元数据注释和标记, 以及数据集的高效访问和筛选。

```
1 frame = pd.DataFrame({'a':np.random.randn(100)})
2 print(frame)
3 store = pd.HDFStore('mydata.h5')
4 # 添加
5 store['obj1'] = frame
6 store['obj1_col'] = frame['a']
7 print(store)
8
9 print(store['obj1'])
10
11 # 两种存储模式 fixed table: 比较慢
12 store.put('obj2',frame,format='table') # 等同于 store['obj1'] = frame
13 print(store)
14
15 # 查询
16 store.select('obj2',where=['index>=10 and index<=15'])
17
18 # 将 DataFrame 数据保存为 HDF5 文件
19 data.to_hdf('test.h5', key='test', mode='w')
20 # 读取
21 pd.read_hdf('mydata.h5','obj3',where=['index<5'])
```

注意:需要安装 tables 模块。使用以下命令来安装 tables:

```
pip install tables -i 镜像源
```


9.3 Web APIs交互(了解)

除了文本数据 例如我们的 csv Excel 这些列数据以外 就是我们的web Api的交互 大多数网站都会通过json或者其他的数据格式 提供一些公用的API接口 那么我们可以通过python 就可以访问这些api

```
1 import requests
2 import pandas as pd
3 url = 'https://weibo.com/ajax/feed/hottimeline?since_id=0&group_id=1028039999&containerid=102803_ctg1_9999_-_ctg1_9999_home&extparam=discover%7Cnew_feed&max_id=0&count=10'
4 resp = requests.get(url)
5 print(resp)
6 # 响应对象的json方法放回一个包含解析过的json字典,加载到python对象当中
7 data = resp.json()
8 # print(data)
9 print(data['statuses'][0])
10 issues = pd.DataFrame(data,columns=['ok','statuses','labels','state'])
11 print(issues)
```

9.4 数据库交互(熟悉)

```

1  # coding: utf-8
2  '''
3  @FileName: 10.sql交互.py
4  @Modify : 2023/10/16 16:35
5  @Description : None
6  '''
7  import pandas as pd
8  from sqlalchemy import create_engine
9
10 # SQL配置信息
11 MYSQL_USER='root'
12 MYSQL_PS = '123456'
13 MYSQL_DB='longzai'
14 MYSQL_HOST='localhost'
15 # 连接配置
16 mysqlengine = create_engine(f"mysql+pymysql://{MYSQL_USER}:{MYSQL_PS}@{MYSQL_HOST}:3306/{MYSQL_DB}?charset=utf8mb4")
17 sql_query = "SELECT * FROM studnet03"
18 df = pd.read_sql(sql_query,mysqlengine)
19 print(df)
20 df['test']=[1,2,3]
21 print(df)
22
23 df.loc[1,'city_id']=87
24 print(df)
25
26 # 第一个参数是哪个表 第二个参数是连接的信息
27 # 第三个 if_exists是否存 append 追加 replace
28 # 第四个 index行索引不要
29 df.to_sql('studnet03',mysqlengine,if_exists='replace',index=False)
30
31 df2 = pd.read_sql(sql_query,mysqlengine)
32 print(df2)
33 df2.loc[3]=[88,888,8888]
34 print(df2)
35 df2.to_sql('studnet03',mysqlengine,if_exists='append',index=False)
36 df3 = pd.DataFrame({
37     "district":[4334],
38     'test':[55]
39 },)
40
41 df3.to_sql('studnet03',mysqlengine,if_exists='append',index=False)

```

10.数据的清洗和准备

首先我们先来看我们的一个数据 来分析一下这写数据🔗二手车之家信息 - 副本.csv

那么对于这些数据我们可以进行数据的加载来加载到我们的python当中

test1.ipynb

Python

```
1 import pandas as pd
2 import numpy as np
3 data = pd.read_csv("二手车之家信息 - 副本.csv")
4 data
```

数据清洗是数据分析关键的一步，直接影响之后的处理工作

数据需要修改吗？有什么需要修改的吗？数据应该怎么调整才能适用于接下来的分析和挖掘？

是一个迭代的过程，实际项目中可能需要不止一次地执行这些清洗操作

处理缺失数据：pd.fillna(), pd.dropna()

龙仔

1.处理缺失数据

```
1 data1 = pd.Series(['a','b',np.nan,'d'])
2 data1
3 data1.isnull()# 返回的是一个series对象的 bool
4 data1[0]=None
5 data1.isnull()# 返回的是一个series对象的 bool
```

- 处理缺失数据：pd.fillna(), pd.dropna()

方法	说明
dropna	根据各标签的值中是否存在缺失数据对轴标签进行过滤，可通过阈值调节对缺失值的容忍度
fillna	用指定值或插值方法（如ffill或bfill）填充缺失数据
isnull	返回一个含有布尔值的对象，这些布尔值表示哪些值是缺失值/NA，该对象的类型与源类型一样
notnull	isnull的否定式

龙仔

过滤/滤除缺失数据

```
1 data1.dropna()
2
3 data1[data1.notnull()]
```

DataFrame

```
1 data2 = pd.DataFrame([[1.,6.5,3.],[1,np.nan,np.nan],[np.nan,np.nan,np.naa],
2 data2
```

```
1 data2.dropna()
2
3 # 丢失全为nan的行
4 data2.dropna(how='all')
5 # 删除列
6 data2.dropna(axis=1,) # 全部删掉了
7
8 data2.dropna(axis=1,how='all')
```

观察参数

```
1 df = pd.DataFrame(np.random.randn(7,3))
2 df.iloc[:4,1]=np.naa
3 df
4 df.iloc[:2,2]=np.nan
5 df
6
7
8 # 删除缺失值等于2 的 缺失值
9 #仅保留至少具有2个非NA值的行
10 df.dropna(thresh=2)
```

填充数据

```
1 df.fillna(0) # 会返回新的对象
2 df1 = df.fillna(0)
3 df1
4 # 不返回新对象 inplace=True即可 就地修改
5
6 df.fillna({1:0.9,2:0}) # 设置初始值
7
```

```
1 df2 = pd.DataFrame(np.random.randn(6,3))
2 df2
3 # 设置缺失值
4 df2.iloc[2:,1]=np.nan
5 df2.iloc[4:,2]=np.nan
6 df2
7
8 df2.fillna(method='ttill')
9 # Pandas dataframe.ffill() 函数用于填充dataframe中的缺失值。
10 # “填充”代表“前向填充”，并将向前传播最后一个有效观察。
11
12
13 # 指定修改的数量
14 df2.fillna(method='ttill',limit=2)
```

2.移除重复数据

```

1  import pandas as pd
2  import numpy as np
3  data = pd.DataFrame({'k1': ['one', 'two']*3+['two'],
4                          'k2': [1,1,2,3,3,4,4]})
5  print(data)
6
7  # 检查是否有重复数据行
8  print(data.duplicated()) # 返回一个bool类型的Series
9
10 # 删除重复数据
11 print(data.drop_duplicates())
12 #是根据全部的列
13
14 data['v1']=range(7)
15 print(data)
16 # 指定列重复删除
17 print(data.drop_duplicates(['k1']))
18
19 # 默认是保存的第一次出现的 删除第二次出现的
20 # 可以指定删除第一次出现的 而保留第二次
21 print(data.drop_duplicates(['k2'],keep='last'))
22
23 # 以上的效果可能不明显我们重新创建一个
24
25 data = pd.DataFrame({'k1': ['one', 'two']*3+['two'],
26                        'k2': [1,1,2,3,3,4,4]})
27 print(data)
28
29 print(data.drop_duplicates(keep='last'))

```

3.利用映射或函数转换数据(重点)

```

1  import pandas as pd
2
3  data = pd.DataFrame({'food': ['Apple', 'banana', 'orange', 'apple', 'Mango', 'tomato'],
4                        'price': [4, 3, 3, 5, 6, 12.3]})
5  data
6  # 添加一列 给对应的数据进行分类
7  # 如果使用添加的操作会非常的麻烦的
8
9  # 所以利用映射和函数来进行
10
11 # 1.先建立一个对应关系
12 meat = {'apple': 'fruit',
13         'banana': 'fruit',
14         'orange': 'fruit',
15         'Mango': 'fruit', # 转小写记得改
16         'tomato': 'vegetables',
17         # 'Apple': 'fruit' # 这个方式标记麻烦
18         }
19
20 # 这个map可以接受一个函数 或者是含有对应关系(映射关系的对象)
21 # map()可以接受一个映射函数或字典,来映射转换Series中的每个值:
22 data['food'].map(meat)
23
24 # 最后一步 将数据中的key转换为小写
25 low = data['food'].str.lower()
26 low
27
28 # 添加一列
29 # data['class'] = data['food'].map(meat)
30 data['class'] = low.map(meat)
31
32 # 利用函数
33 data['class1'] = data['food'].map(lambda x: meat[x.lower()])
34
35
36
37 df = pd.DataFrame({'A': [-1, 2, 3], 'B': [4, 5, 6], 'C': [4.4, 5.6, 5.3]})
38
39
40 # apply()可以应用一个函数至Series或DataFrame的每列/行,实现转换:
41 # 对每列应用len函数 长度
42 print(df.apply(len))
43
44 # 对每行应用求和函数
45 print(df.apply(lambda x: x.sum(), axis=1))
46 # 最大

```

```
47 print(df.apply(lambda x: x.max(), axis=1))
48 # 最小
49 print(df.apply(lambda x: x.min(), axis=1))
50 # 求平方
51 print(df.apply(lambda x: x.pow(2), axis=1))
52 # 求绝对值
53 print(df.apply(lambda x: x.abs(), axis=1))
54 # 四舍五入
55 print(df.apply(lambda x: x.round(), axis=1))
56 ...
57
58 apply()会遍历Series/DataFrame,并将每一元素/行/列传给函数执行。可以通过axis参数控制
   是逐列还是逐行应用。
59 map()/apply()提供了在pandas中灵活转换数据的方式。需要对数据集抽象执行操作时可以活用它
   们。
60 ...
```

4.替换值


```

1  import pandas as pd
2  import numpy as np
3  data = pd.Series([1,-999,2,-1000,3])
4  data
5
6  data.loc[data < 3] = 0
7  print(data.map(lambda x: x * 100))
8  print(data.apply(lambda x: x * 100))
9  data = pd.DataFrame(data)
10 print(data)
11 data[0] = data.map(lambda x: x * 100)
12 data[0] = data.apply(lambda x: x * 100)
13
14
15 df = data.where(data > 0, -1)
16 '''
17 - to_replace: 要替换的值或者是一个字典, 指定了替换规则
18 - value: 替换后的新值
19 - inplace: 是否在原地进行替换, 默认为False
20 '''
21 data.replace(-999,np.nan) # 返回一个新的Series对象
22 # 替换多个数据
23 data.replace([-999,-1000],np.nan)
24
25 # 替换成不同的值
26 data.replace([-999,-1000],[np.nan,0])
27 # 除了列表 还可以是字典
28 data.replace({-999:np.nan,-1000:0})
29
30 data1 = data.replace([-999,-1000],[np.nan,0])
31 data
32

```

5.重命名轴索引

```

1 data = pd.DataFrame(np.arange(12).reshape((3,4)),
2                     index=['BeiJing', 'Tokyo', 'New York'],
3                     columns=['one', 'two', 'three', 'four'])
4 data
5
6 # 重新索引
7 data.reindex(['BeiJing', 'New York', 'Tokyo'])
8 data
9
10 # 利用重新索引改为abc能不能呢
11 data.reindex(['a', 'b', 'c'])# 可以发现是没有任何数据的 这是我们使用reindex需要注意的
12 # 其次 这个reindex他所以操作的是一个新的对象 返回的也是新的对象 不会对原来的时间影响
13
14 # 大写
15 tran = lambda x:x[:4].upper()
16 data.index.map(tran)
17
18 data.index = data.index.map(tran)
19 data
20
21 # rename 不修改源数据的情况下
22 # str.title()方法把每个单词的首字母转成大写了。
23 # str.upper 会将索引或列名全部转为大写字母。
24 data.rename(index = str.title, columns = str.upper)
25 data
26
27 # 结合字典对象 对标签进行更新 替换
28 data.rename(index={'TOKY':'东京'}, columns={'three':'第三年'})
29
30
31 data
32 # 就地修改DataFrame对象
33 data.rename(index={'TOKY':'东京'}, columns={'three':'第三年'}, inplace=True)

```

6.离散化和面元化分

为了方便数据分析

1. 什么是离散化 (Discretization)

离散化(Discretization)是一种对连续性数据进行处理的方法,目的是将连续域转化为离散域,也就是将连续变量转变为离散变量。

它的主要作用有:

1. 减少数据的维度,降低存储和计算成本。
2. 消除离散数据中不存在的噪声。
3. 使得数据可用于特定的分析算法。某些算法要求变量为离散的。
4. 提高分析结果的可理解性

将连续变量转化为离散变量的过程

```
1  # coding: utf-8
2
3  """
4  @FileName: 6.离散化和面元化分.py
5  @Modify : 2023/10/24 13:59
6  @Author : 水不在深 有龙则灵
7  """
8  import numpy as np
9
10 # 生成待离散化的连续数据
11 data = np.arange(1,100)
12 print(data)
13
14 # 设置切分的区间数量
15 n_bins = 5
16
17 # 计算数值范围
18 min_val,max_val = np.min(data),np.max(data)
19 print(min_val)
20 print(max_val)
21
22 # 计算每个区间的宽度
23 bin_width = (max_val-min_val)/n_bins
24 # 利用np.digitize 等宽离散化
25 bins = np.arange(min_val,max_val,bin_width)
26 print(bins)
27
28
29 discrete_data = np.digitize(data,bins)
30 print(discrete_data)
31 print(data[1])
```

2. 什么是面元 (Binning) 化分

是一种数据处理技术，将连续数据划分为离散的区间或面元，用于更好地理解数据的分布特征、消除噪声、处理缺失值或减少数据的复杂性。

主要目的是将连续数据转化为离散的类别或区间，以便进一步分析、建模或可视化。通过将连续数据进行离散化处理，可以减少数据的复杂性，并提供更简洁、易解释的表示形式。

是将连续变量离散化的过程。它将一个连续的变量转换成有限个数的面元(bin)。每个面元都包含一个值范围,通过这个过程可以降低变量的复杂度

```
1  # coding: utf-8
2
3  """
4  @FileName: 7.离散化和面元化分.py
5  @Modify : 2023/10/19 17:14
6  @Author : 水不在深 有龙则灵
7  """
8  import pandas as pd
9  import numpy as np
10 # 生成数据样本
11 ages = [19,22,25,27,28,29,37,31,61,45,32,91]
12 # 指定面元区间端点
13 bins = [18,25,35,60,100]
14 # 面元化
15 cats = pd.cut(ages,bins)
16 print(cats)
17 print(cats.codes) # 打印面元化后的代码
18 print(cats.categories) # 打印面元化的区间范围
19 # 设置面元的左右开闭
20 print(pd.cut(ages,bins,right=False))
21 # 自定义面元的名称
22 names = ['青年','年轻人','中年人','老年人']
23 print(pd.cut(ages,bins,labels=names))
24
25 data = np.random.randn(1000)
26 print(data)
27 print(pd.cut(data, 4,precision=2))
28 # 不指定区间,按数据样本数量等分面元
29
30 # 手动指定
31 cats = pd.cut(data,[0,0.1,0.5,0.9,1.])
32 print(cats)
33 print(pd.Series(cats).value_counts())
34
35 cats = pd.qcut(data,[0,0.1,0.5,0.9,1.])
36 print(pd.Series(cats).value_counts())
37 # 从结果我们可以看到:
38 # 1. 数据被离散化到了4个面元(bin)中,比如(-1.423, -0.0594]就是一个面元
39 # 2. 每个面元区间都包含了数据样本的数量,比如第一个面元包含400个样本
40 # 3. 面元区间是不相交的,并且覆盖了全部的数据范围
41 # 所以这个结果展示了原始数据经过离散化后的数据分布情况。通过面元化将连续数据转换成了有限的离散区间。
42 # 每个面元区间其实就是一个类别,统计每个面元中的样本数量
```

7.检测和过滤异常值

```
1 import pandas as pd
2 import numpy as np
3 data = pd.DataFrame(np.random.randn(1000,4))
4 print(data)
5 # 找到绝对值大于2的数据
6 # print(data[(np.abs(data)) > 2].all(1,axis=1))
7 cond = (np.abs(data) > 2 ).any(axis=1)
8 print(data.loc[cond])
9 data.loc[cond]=3
10
11
12
13 # # 使用abs()计算绝对值
14 abs_df = df.abs()
15 # 找出大于3的布尔索引
16 bool_idx = abs_df > 2
17 pos = np.where(bool_idx)
18 idx_dict = {}
19 for r, c in zip(pos[0], pos[1]):
20     idx_dict[r] = c
21
22 print(idx_dict)
23 for i in idx_dict.keys():
24     df.iloc[i,idx_dict[i]]=3
25 print(df)
26
27
28 print(data.describe())# 表述统计分析
29 # 使用describe()了解数据的统计分布,如mean、std等,根据统计值判断异常值
```

8.排列和随机采样

```
1 pf = pd.DataFrame(np.arange(5*4).reshape(5,4))\
2 df
3 # 新数据数组
4 sam = np.random.permutation(5)
5 sam
6 # 排列
7 df.take(sam)
8
9
10 # 如果不想用上面的 也可以使用 smaple
11
12 # 随机采样 子集
13 df.smaple(n=3)
14
15 # 通过替换来参数随机样本
16 ch = pd.Series([5,7,1,6,3])
17 ch
18
19 # 重复选择 画图
20 ch.sample(n=10, replace=True)
21
```

9.字符串对象方法

1. 对象方法

```

1  val = ' a,B,c '
2
3  # 去除空白包括\n
4  p = [x.strip() for x in val.split(',')]
5  print(p)
6
7  f,s,t = p
8
9  # 字符串加法
10 print(f+'::'+s+'::'+t)
11
12 # 3. 字符串拼接
13 # - ':'.join(list) - 使用':'拼接列表中的元素
14 # - str1 + str2 - 使用+拼接两个字符串
15 print('::'.join(p))
16
17 # 4. 检查包含性
18 # - 'a' in str - 检查a是否包含在字符串str中
19 print(p)
20 print("-----")
21 print('p' in p)
22
23 # 5. 查找子串
24 # - str.index(sub) - 返回sub子串的第一个索引
25 # - str.find(sub) - 返回sub子串的第一个索引,如果不存在返回-1
26 print(val.index(','))
27 print(val.find('::'))
28
29 print(val.replace(',', '::'))
30 #
31 print(val)
32 print("-----")
33 #
34
35 # 9. 去除空白字符
36 # - str.strip() - 去除两端空白
37 # - str.lstrip() - 去除左端空白
38 # - str.rstrip() - 去除右端空白
39 # 通过指定的分隔符将字符串拆分为一组子串
40 print(val.rstrip())
41 print(val.lstrip())
42 print(val.strip())
43 # 1. 分割字符串
44 # - str.split(sep) - 使用sep作为分隔符拆分字符串,返回列表
45 # - str.rsplit(sep) - 从右侧开始拆分
46 list1 = val.strip(',')
47 print(list1)

```

```
48
49 # 7. 大小写转换
50 # - str.lower() - 全部转换为小写
51 # - str.upper() - 全部转换为大写
52 # 分别将字母字符转换为小写或大写
53 print(val.lower())
54 print(val.upper())
55
56 # 8. 填充对齐
57 # - str.ljust() - 左侧填充
58 # - str.rjust() - 右侧填充
59 # 用空格（或其他字符）填充字符串的空白侧以返回符合最低宽度的字符串
60
61 print(val.ljust(9,"+"))
62 print(val.rjust(4,"--"))
63
64
```

10.正则表达式


```

1  # 导入re模块
2  import re
3
4
5  string = "123 hello 123 world 456"
6  # 创建正则表达式
7  pattern = r'\d+'
8  # 1. match匹配开始位置
9  match = re.match(pattern, string)
10 print(match) # 匹配到123
11 # 2. search搜索整个字符串
12 search = re.search(pattern, string)
13 print(search.group()) # 匹配到123
14
15 # 3. findall查找所有匹配
16 all = re.findall(pattern, string)
17 print(all) # ['123', '456']
18
19 # 4. split分割字符串
20 split = re.split(pattern, string)
21 print(split) # ['hello ', ' world ']
22
23 # 5. sub替换匹配
24 replaced = re.sub(pattern, 'NUMBER', string)
25 print(replaced) # hello NUMBER world NUMBER
26
27 '''
28 - ( ) 定义了数字组的捕获
29 - \d+ 用来匹配数字
30 - - 用来匹配固定的分隔符
31 '''
32 pattern = r'(\d+)-(\d+)-(\d+)'
33 string = "2021-01-01"
34 match = re.match(pattern, string)
35
36 print(match.group(0)) # 整个匹配内容 2021-01-01
37 print(match.group(1)) # 第一个括号匹配部分 2021
38 print(match.group(2)) # 第二个括号匹配部分 01
39 print(match.group(3)) # 第三个括号匹配部分 01

```

11.pandas的矢量化字符串函数

矢量化字符串函数是指pandas中可以对Series或DataFrame整列字符串进行操作的字符串方法。其特点是：

1. 矢量化:将字符串操作应用到整列上,而不是针对单个元素进行迭代操作。
2. 函数化:直接通过Series/DataFrame调用方法的方式使用,而不是for循环。
3. 加速运算:利用了C/Cython、NumPy等高速计算库的运算能力。
4. 减少代码量:方法链式调用,避免繁琐的循环操作。
5. 提高可读性:直观的方法命名,代码逻辑更清晰。

简单来说,矢量化字符串函数就是pandas提供的一组用于高效字符串处理的函数式方法。

优点是:

- 简化代码,不需要for循环针对单个元素逐一操作
- 更高效,调用的是优化后的C/Fortran函数
- 更清晰易读的链式调用

```
1 data = {'a':'day@qq.com','b':'steve@gmail.com','c':'sam@gamil.com','d':np.  
nan}  
2 data = pd.Series(data)  
3 print(data)  
4  
5  
6 # 缺失值检查  
7 data.isnull()  
8  
9 # 按照@分割  
10 data.map(lambda x:x.split('@'))  
11  
12 # 判断是否包含子字符串  
13 data.str.contains('i')  
14  
15 # 替换指定子字符串  
16 data.str.replace('i', 'I')  
17  
18 # 按空格分割成列表  
19 data.str.split()  
20  
21 # 提取第一个元素  
22 data.str.get(0)  
23  
24 # 截取前3个字符  
25 data.str.slice(0, 3)  
26  
27 # 使用正则表达式提取名字  
28 data.str.extract('([A-Za-z]+)')  
29  
30 # 矢量化元素查找  
31 data.str.findall('@')  
32  
33 # 切片切割  
34 data.str[:5]
```

12.总结

```

1 data = pd.read_csv('')
2 data
3
4 data.isnull()
5
6 (data.isnull()).sum()
7
8 # 根据那一列删除
9 data.dropna(subset=['xxx'],inplace=True)
10 data
11
12 # 检查重复值
13 data.duplicated()
14 # 重复值统计
15 data.duplicated().sum()
16
17 data.drop_duplicates(inplace=True)
18 len(data)
19
20 # 字符串操作
21 (data.列.str.contains('万')).sum()
22 # 删除万
23 data.价格.map(lambda x:x.replace('万',''))
24 # 数据类型 方式一
25 data.价格.map(lambda x:float(x.replace('万','')))
26 data['列'] = data.shoujia.map(lambda x:float(x.replace('万','')))
27 # 方式二
28 data.价格.map(lambda x:replace('万','')).astype('float64')
29
30
31 # 处理异常数据
32 data.drop([4,9],inplace=True)
33 #打印前五
34 data.head()
35
36

```

13.案例

📎车行信息-副本.csv

数据清洗案例：

1. 去除重复值
2. 重命名列名(何以修改为中文，也可以是专业英语)

3. 删除异常值
4. 年份，售价所在列的数据进行转换（数值型数据）
5. 缺失值处理

```
1 import pandas as pd
2 df = pd.read_csv('车行信息-副本.csv')
3 df
4 # 去除重复值
5 df.drop_duplicates(keep=False,inplace=True)
6 # 重命名列名(何以修改为中文，也可以是专业英语)
7 df.columns = ['汽车型号','图片','售价','上牌时间','表显里程(公里)','链接','地区',
8               '来源']
9 df
9 # 删除异常值
10 s = df['汽车型号']
11 mask = s.str.contains('x')
12 df = df.loc[~mask]
13 # 年份，里程，售价，原价所在列的数据进行转换（数值型数据）
14 df['上牌时间'] = pd.to_datetime(df['上牌时间'],format='%Y/%m')
15 df['售价'] = df.售价.map(lambda x:float(x.replace('万','')))
16 df
17 # 缺失值处理
18 df.dropna(how='all')
```

11.数据规整

1.层次化索引

```

1  import numpy as np
2  import pandas as pd
3
4  # 创建一个多重索引的Series
5  data = pd.Series(np.random.randn(9),
6                  index=[['a', 'a', 'a', 'b', 'b', 'c', 'c', 'd', 'd'],
7                        [1, 2, 3, 1, 3, 1, 2, 2, 3]])
8
9  # 显示Series
10 print(data)
11
12 # 选择外层索引为'a'的子集
13 print(data['a'])
14
15 # 切片,选择'b'至'c'索引之间的子集
16 print(data['b':'c'])
17
18 # 通过loc选择多重索引的子集,选择外层索引为'b','d'的行
19 print(data.loc[['b', 'd']])
20
21 # 通过loc选择多重索引的子集,选择内层索引为2的元素
22 print(data.loc[:, 2])
23
24
25 frame = pd.DataFrame({'a': range(7), 'b': range(7, 0, -1),
26                      'c': ['one', 'one', 'one', 'two', 'two', 'two', 'two'],
27                      'd': [0, 1, 2, 0, 1, 2, 3]})
28
29 # 显示DataFrame
30 print(frame)
31
32 # 设置多重索引,外层索引为'c','d',内层保持原索引
33 frame2 = frame.set_index(['c', 'd'])
34
35 # 显示设置了多重索引的DataFrame
36 print(frame2)
37
38 # 设置多重索引,外层索引为'c','d',但保留原索引列
39 # frame2 = frame.set_index(['c', 'd'], drop=False)
40
41 # 重置索引,移除多重索引,恢复原索引
42 print(frame2.reset_index())
43

```

2. 数据链接

```

1 left=pd.DataFrame({'key':['K0','K1','K2','K3'],
2                     'A':['A0','A1','A2','A3'],
3                     'B':['B0','B1','B2','B3']})
4 right= pd.DataFrame({'key':['K0','K1','K2','K3'],
5                       'C':['C0','C1','C2','C3'],
6                       'D':['D0','D1','D2','D3']})
7 left
8 right
9
10 # 数据链接
11 pd.merge(left,right)
12

```

3.数据合并(pd.merge)

```

1 left=pd.DataFrame ( ('key':['ko','K1','K2','K3'],
2                       'A':['A0','A1','A2','A3'],
3                       'B':['B0','B1','B2','B3'])
4 right=pd.DataFrame( ('key':['K0','K1','K2','K3'],
5                       'C':['C0','C1','C2','C3'],
6                       'D':['D0','D1','D2','D3']})
7 left
8 right
9 pd.merge(left,right)

```

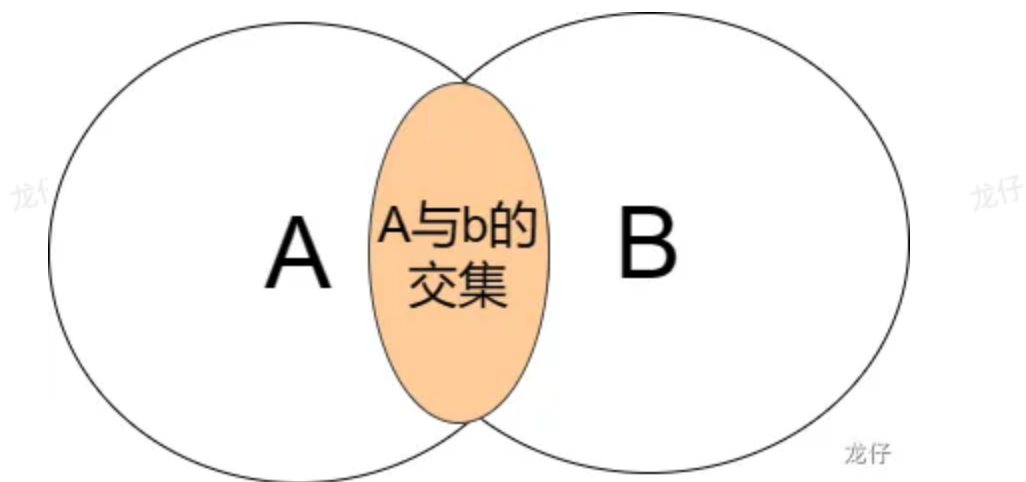
- pd.merge
- 根据单个或者多个键将不同DataFrame的行链接起来
- 类似数据库的连接操作

```

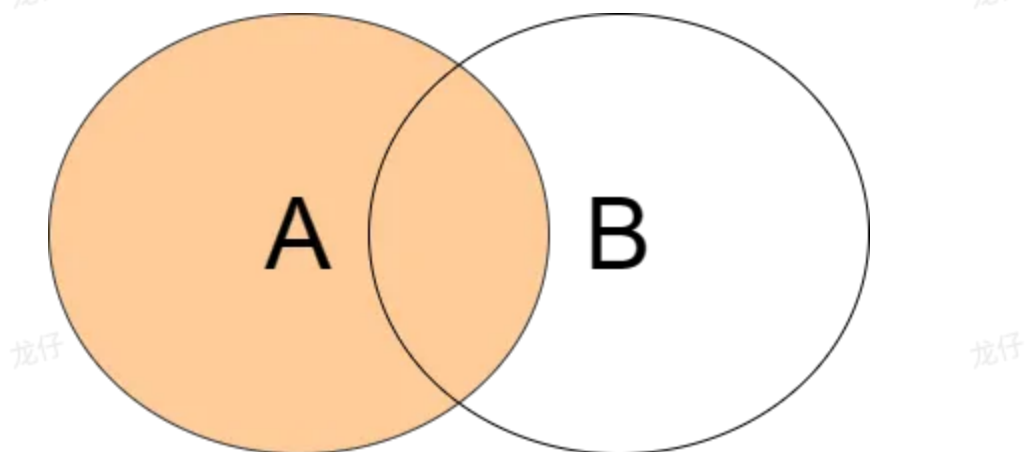
1 merge(left, right, how='inner', on=None, left_on=None, right_on=None,
2        left_index=False, right_index=False, sort=True,
3        suffixes=('_x', '_y'), copy=True, indicator=False)

```

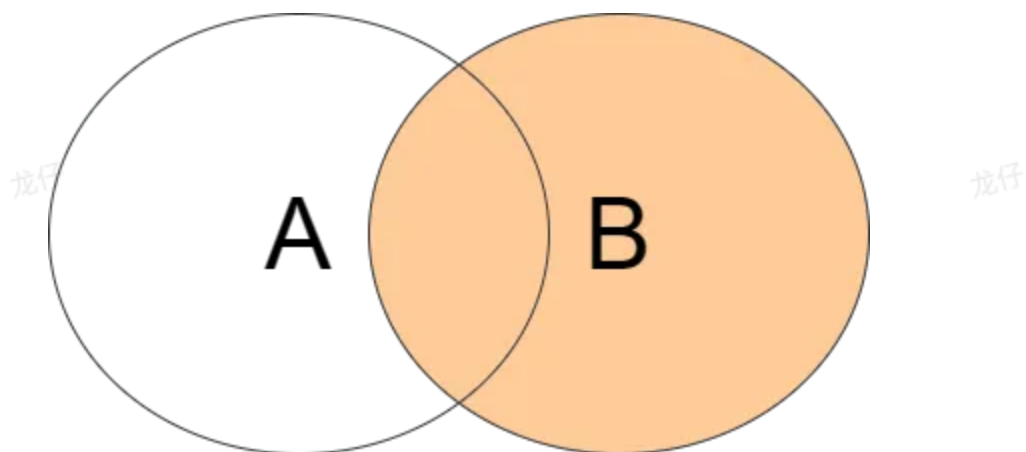
- 参数介绍
 - left和right: 两个不同的DataFrame;
 - how: 连接方式, 有inner、left、right、outer, 默认为inner;
 - 内链接inner 对两张表都有的键的交集进行联合



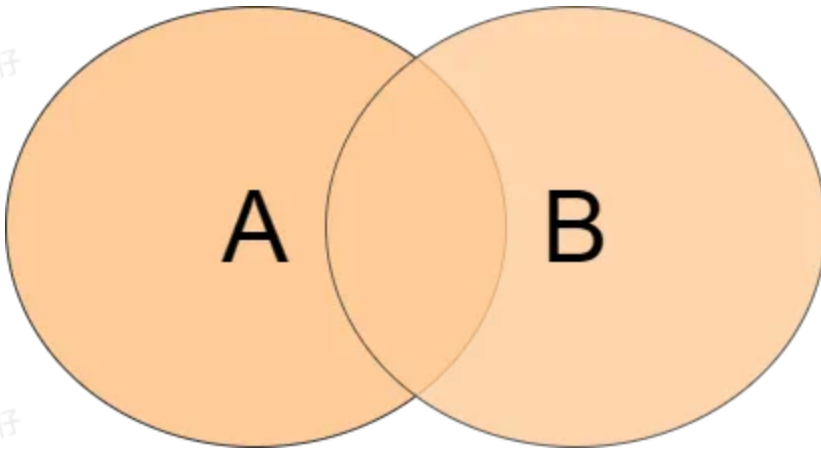
- 左链接left:对所有左表的键进行联合



- 右链接right:对两张表都有的键的交集进行联合



- outer:全链接: 对两者表的都有的键的并集进行联合



- on: 指的是用于连接的列索引名称，必须存在于左右两个DataFrame中，如果没有指定且其他参数也没有指定，则以两个DataFrame列名交集作为连接键；

```
1 left=pd.DataFrame({'key':['ko','K1','K2','K3'],
2                     'A':['A0','A1','A2','A3'],
3                     'B':['B0','B1','B2','B3']})
4 right=pd.DataFrame({'key':['K0','K1','K2','K3'],
5                      'C':['C0','C1','C2','C3'],
6                      'D':['D0','D1','D2','D3']})
7 left
8 right
9 pd.merge(left,right,on='key') # 画图
10
11 # 也可以指定多个键
12 import pandas as pd
13 # 也可以指定多个键
14 left=pd.DataFrame({'key1':['K0','K0','K1','K2'],
15                    'key2':['k0','K1','K0','K1'],
16                    'A':['A0','A1','A2','A3'],
17                    'B':['B0','B1','B2','B3']})
18 right=pd.DataFrame({'key1':['K0','K1','K1','K2'],
19                     'key2':['k0','K0','K0','K0'],
20                     'C':['C0','C1','C2','C3'],
21                     'D':['D0','D1','D2','D3']})
22 print(left)
23 print(right)
24
25 print(pd.merge(left, right, on=['key1','key2']))
```

left					right					Result						
	A	B	key1	key2		C	D	key1	key2		A	B	key1	key2	C	D
0	A0	B0	K0	K0	0	C0	D0	K0	K0	0	A0	B0	K0	K0	C0	D0
1	A1	B1	K0	K1	1	C1	D1	K1	K0	1	A2	B2	K1	K0	C1	D1
2	A2	B2	K1	K0	2	C2	D2	K1	K0	2	A2	B2	K1	K0	C2	D2
3	A3	B3	K2	K1	3	C3	D3	K2	K0							

龙仔

```

1 # 也可以指定多个键
2 import pandas as pd
3 # 默认是内链接 当然也可以设置其他的 链接方式
4 left=pd.DataFrame({'key1':['K0','K0','K1','K2'],
5                     'key2':['k0','K1','K0','K1'],
6                     'A':['A0','A1','A2','A3'],
7                     'B':['B0','B1','B2','B3']})
8 right=pd.DataFrame({'key1':['K0','K1','K1','K2'],
9                     'key2':['k0','K0','K0','K0'],
10                    'C':['C0','C1','C2','C3'],
11                    'D':['D0','D1','D2','D3']})
12 print(left)
13 print(right)
14
15 print(pd.merge(left, right, on=['key1','key2']))
16 left=pd.DataFrame({'key1':['K0','K0','K1','K2'],
17                     'key2':['k0','K1','K0','K1'],
18                     'A':['A0','A1','A2','A3'],
19                     'B':['B0','B1','B2','B3']})
20 right=pd.DataFrame({'key1':['K0','K1','K1','K2'],
21                     'key2':['k0','K0','K0','K0'],
22                    'C':['C0','C1','C2','C3'],
23                    'D':['D0','D1','D2','D3']})
24 print(left)
25 print(right)
26
27 print(pd.merge(left, right,how="left/right", on=['key1','key2'])) # 画
图

```

left					right					Result						
	A	B	key1	key2		C	D	key1	key2		A	B	key1	key2	C	D
0	A0	B0	K0	K0	0	C0	D0	K0	K0	0	A0	B0	K0	K0	C0	D0
1	A1	B1	K0	K1	1	C1	D1	K1	K0	1	A1	B1	K0	K1	NaN	NaN
2	A2	B2	K1	K0	2	C2	D2	K1	K0	2	A2	B2	K1	K0	C1	D1
3	A3	B3	K2	K1	3	C3	D3	K2	K0	3	A2	B2	K1	K0	C2	D2
										4	A3	B3	K2	K1	NaN	NaN

龙仔

```

1 # 也可以指定多个键
2 import pandas as pd
3 # 默认是内链接 当然也可以设置其他的 链接方式
4 left=pd.DataFrame({'key1':['K0','K0','K1','K2'],
5                     'key2':['k0','K1','K0','K1'],
6                     'A':['A0','A1','A2','A3'],
7                     'B':['B0','B1','B2','B3']})
8 right=pd.DataFrame({'key1':['K0','K1','K1','K2'],
9                     'key2':['k0','K0','K0','K0'],
10                    'C':['C0','C1','C2','C3'],
11                    'D':['D0','D1','D2','D3']})
12 print(left)
13 print(right)
14
15 print(pd.merge(left, right, on=['key1','key2']))
16 left=pd.DataFrame({'key1':['K0','K0','K1','K2'],
17                     'key2':['k0','K1','K0','K1'],
18                     'A':['A0','A1','A2','A3'],
19                     'B':['B0','B1','B2','B3']})
20 right=pd.DataFrame({'key1':['K0','K1','K1','K2'],
21                     'key2':['k0','K0','K0','K0'],
22                    'C':['C0','C1','C2','C3'],
23                    'D':['D0','D1','D2','D3']})
24 print(left)
25 print(right)
26
27 print(pd.merge(left, right,how="outer", on=['key1','key2'])) # 画图

```

龙仔

left					right					Result						
	A	B	key1	key2		C	D	key1	key2		A	B	key1	key2	C	D
0	A0	B0	K0	K0	0	C0	D0	K0	K0	0	A0	B0	K0	K0	C0	D0
1	A1	B1	K0	K1	1	C1	D1	K1	K0	1	A1	B1	K0	K1	NaN	NaN
2	A2	B2	K1	K0	2	C2	D2	K1	K0	2	A2	B2	K1	K0	C1	D1
3	A3	B3	K2	K1	3	C3	D3	K2	K0	3	A2	B2	K1	K0	C2	D2
										4	A3	B3	K2	K1	NaN	NaN
										5	NaN	NaN	K2	K0	C3	D3

龙仔

- left_on: 左侧DataFrame中用于连接键的列名，这个参数左右列名不同但代表的含义相同时非常的有用；
- right_on: 右侧DataFrame中用于连接键的列名；
- left_index: 使用左侧DataFrame中的行索引作为连接键；
- right_index: 使用右侧DataFrame中的行索引作为连接键；

按照索引进行链接:

```

1 # 按索引连接
2 df_obj1=pd.DataFrame('key':['b','b','a','c','a','a','b'],
3 'data1':np.random.randint(0,10,7))
4 df_obj2=pd.DataFrame(('data2':np.random.randint(0,10,3)),index=['a','b','d'
5 ])
6 print(pd.merge(df_obj1,df_obj2,left_on='key',right_index=True))

```

- sort: 默认为True，将合并的数据进行排序，设置为False可以提高性能；
- suffixes: 字符串值组成的元组，用于指定当左右DataFrame存在相同列名时在列名后面附加的后缀名称，默认为('_x', '_y')；

处理重复列名

```

1 suffixes,默认为_x,_y

```

示例代码

```

1  #处理重复列名
2  df_obj1 = pd.DataFrame('key':["b",b', 'a', 'c', 'a', 'a', 'b'], 'data':np.random.
    randint(0,10,7))
3  df_obj2=pd.DataFrame(('key': ['a', 'b', 'd'], 'data':np.random.randint(0,10,3))

4  df_obj1
5  df_obj2
6  print(pd.merge(df_obj1,df_obj2,on='key',suffixes=('_left','_right')))
```

- copy: 默认为True, 总是将数据复制到数据结构中, 设置为False可以提高性能;
- indicator: 显示合并数据中数据的来源情况

join合并 二

```

1  left2=pd.DataFrame([[1.,2.],[3.,4.],[5.,6.]],
2                      index=['a','c','e'],
3                      columns=['语文','数学'])
4  riht2=pd.DataFrame([7.,8.],[9.,10.1],[11.,12.1],[13,141],
5                      index=['b','c','d','e'],
6                      columns=(['英语','综合']))
7  left2
8  right2
9  pd.merge(left2,right2,how="outer",left_index=True,right_index=True)
10
11 # join是按照索引进行合并 join不能有重叠的列的
12 left2.join(right2,how="outer")
13 left2=pd.DataFrame([1.,2.,3.,4.],[5.,6.],
14                    index=['a','c','e'],
15                    columns=['语文','数学'])
16 riht2=pd.DataFrame([7.,8.],[9.,10.1],[11.,12.1],[13,141],
17                    index=['b','c','d','e'],
18                    columns=(['语文','综合']))
19 left2.join(right2,how="outer") # 报错
```

concat

沿轴方向将多个对象合并到一起

1. Numpy的concat

np.concatenate

示例代码:

```

1 import numpy as np
2 import pandas as pd
3 arr1 = np.random.randint(0,10,(3,4))
4 arr2 = np.random.randint(0,10,(3,4))
5 print(arr1)
6 print(arr2)
7 print(np.concatenate([arr1,arr2]))
8 print(np.concatenate([arr1,arr2],axis=1))

```

2. pd.concat

1. 注意指定轴方向，默认axis=0
2. join指定合并方式，默认为outer
3. Series合并时查看行索引有无重复

```

1 df1=pd.DataFrame(np.arange(6).reshape(3,2),index=list('abc'),columns=['one','two'])
2 df2=pd.DataFrame(np.arange(4).reshape(2,2)+5,index=list('ac'),columns=['three','four'])
3 df1
4 df2
5
6
7 pd.concat([df1,df2])#默认外连接，axis=0
8
9 pd.concat([df1,df2],axis="columns")#指定axis=1连接
10
11 #同样我们也可以指定连接的方式为inner
12 pd.concat([df1,df2],axis=1,join="inner")

```

4.重塑层次化索引

1. 重塑(Reshaping)层次化索引(Hierarchical Index)是指改变DataFrame的形状,同时保留原有的层次化索引结构的操作。
2. 在Pandas中，重塑层次化索引（也称为多级索引）通常涉及**stack()**和**unstack()**函数。
stack()函数将数据的列“压缩”成行，增加层次化索引的深度，而**unstack()**函数将层次化索引的最内层转移到列，减少索引的深度。

例如，**unstack()**可以将具有多级索引的DataFrame转换为更传统的表形式，其中每个索引级别变成表的列。相反，**stack()**将DataFrame的列转换为行，形成一个具有多级索引的Series。

```

1  import pandas as pd
2  import numpy as np
3
4  # 创建一个DataFrame, 索引是学生的名字, 列是科目。
5  data = pd.DataFrame(np.arange(6).reshape((2,3)),
6                      index=pd.Index(['老王', '小刘'], name='姓名'),
7                      columns=pd.Index(['语文', '数学', '英语'], name='科目'))
8
9  print(data)
10
11 # stack: 将DataFrame的列“叠加”成Series, 同时列名会成为层次化索引的第二级。
12 # 使用stack()将DataFrame的列转换为具有多级索引的Series。
13 r = data.stack()
14 print(r)
15
16 print(type(r))
17
18 '''
19 将层级索引展开
20 Series-->DataFrame
21 默认操作内层索引
22 可以通过level指定操作索引的级别
23 '''
24 # 使用unstack()将多级索引的Series重新转换成DataFrame。
25 print(r.unstack())
26 #unstack: 将Series的层次化索引“展开”成DataFrame的列, 原索引的级别将成为列的层次化索引。
27
28 # 使用unstack(level=0)或unstack('姓名')来指定要展开的索引级别。
29 print(r.unstack(level=0)) # 展开内层, 相当于默认unstack
30 print(r.unstack('姓名')) # 通过名称指定展开外层索引
31
32 # 使用unstack(fill_value=0)来用0填充缺失值。
33 s.unstack(fill_value=0)
34
35 # 创建两个Series,
36 a1 = pd.Series(np.arange(4), index=list('abcd'))
37 a2 = pd.Series([4,5,6], index=list('cde'))
38 # 水平连接两个Series
39 ps = pd.concat([a1, a2], keys=['data1', 'data2'])
40
41
42 print(ps)
43 print(type(ps)) # Series
44 # 对其中一个Series进行unstack()操作, 显示缺失值,
45 print(s1.unstack()) # 有缺失值
46

```

```
47 # 然后再使用stack()将其还原，展示了默认的丢弃缺失数据的行为。
48 print(s1.unstack().stack())
49 # 默认过滤缺失数据
50
51 # 不过滤缺失值
52 print(s1.unstack().stack(dropna=False))
```

5.轴向旋转(了解)

在Pandas中，你可以使用**pivot**和**pivot_table**函数来进行轴向旋转，以重新组织DataFrame中的数据。这可以帮助你更好地理解和分析数据。以下是一些有关如何进行轴向旋转的基本信息：

轴向旋转的好处是可以重排数据,获得更适合分析的形式,提高后续处理、清洗等的效率。

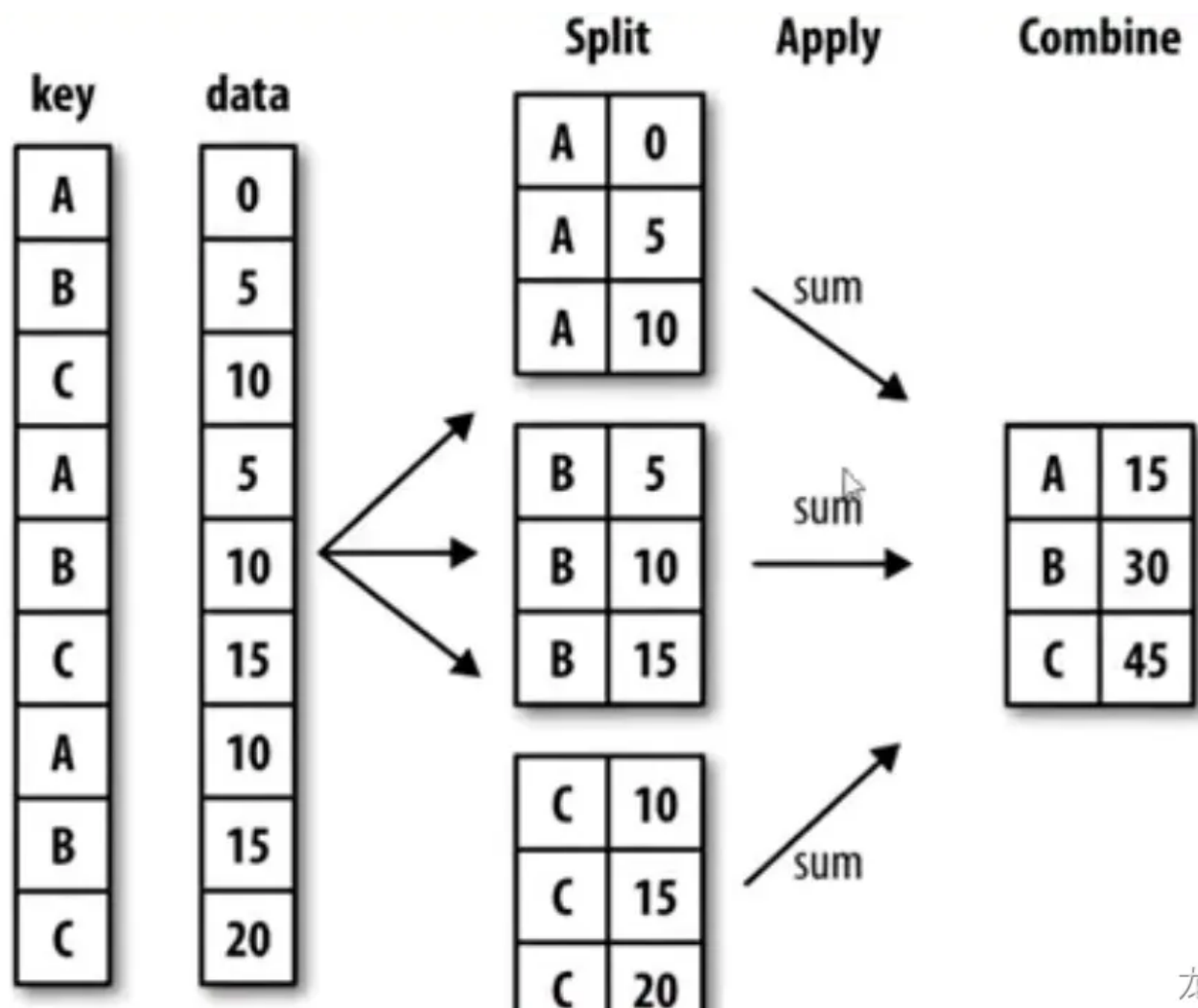

```

1 df3 = pd.DataFrame({'date': ['2023-11-22', '2023-11-22', '2023-11-22', '2023-11-22', '2023-11-22'],
2                       'class': ['a', 'b', 'b', 'c', 'c'],
3                       'values': [5, 3, 2, 6, 1]}, columns=['date', 'class', 'values'])
4 df3
5
6 # pivot函数: pivot函数用于将DataFrame中的列转换为行索引, 将行索引转换为列,
7 # 并重新安排数据。它的基本语法如下:
8 # df.pivot(index='列索引', columns='行索引', values='值')
9 # 其中, '列索引'指定了新DataFrame中的行索引, '行索引'指定了新DataFrame中的列, '值'是要填充的值。
10 df3.pivot(index='date', columns='class', values='values')
11 # 没有值 写得到层次化索引
12 df3.pivot(index='date', columns='class')
13
14
15 # 通过另外一种方式 其实pivot 就是set_index和 unstack 结合的使用的一种方式
16 df3.set_index(['date', 'class']).unstack('class')
17 # 在这里给大家将重塑和轴向 主要是想 告诉大家pandas中的对数据的重排操作
18 # 那么重排操作的好处就是获得想要的数据的格式 这样方便好进行清洗和处理
19
20 # pivot_table函数: pivot_table函数与pivot函数类似,
21 # 但可以处理重复的索引对应多个值的情况, 并提供了一些聚合函数来处理重复的值。它的基本语法如下:
22 # pd.pivot_table(data, values='值', index='列索引', columns='行索引', aggfunc='聚合函数')
23 # 其中, '聚合函数'是可选参数, 用于聚合重复的值。
24 import pandas as pd
25
26 data = {'日期': ['2022-01-01', '2022-01-01', '2022-01-02', '2022-01-02'],
27         '城市': ['New York', 'Los Angeles', 'New York', 'Los Angeles'],
28         '温度': [32, 75, 35, 70]}
29 df = pd.DataFrame(data)
30
31 pivot_table_df = pd.pivot_table(df, values='温度', index='日期', columns='城市', aggfunc='mean')
32 print(pivot_table_df)
33

```

12.数据分组和聚合

```
1 df = pd.read_csv('xxx')
2 df.head()
```



龙仔

```

1  import pandas as pd
2
3  df1 = pd.DataFrame({'fruit': ['apple', 'banana', 'orange', 'apple', 'banana'],
4                          'color': ['red', 'yellow', 'yellow', 'cyan', 'cyan'],
5                          'price': [8.5, 6.8, 8.6, 7.8, 6.4]})
6  print(df1)
7
8  # print(df1.groupby(by='fruit'), type(df1.groupby(by='fruit')))
9  # 按'fruit'列进行分组
10 g = df1.groupby(by='fruit')
11 # df1.groupby(by='fruit') 返回的是一个 DataFrameGroupBy 对象。这是一个特殊类型的
    对象,
12 # 它允许你按照指定的列 (在这种情况下是'fruit'列) 对DataFrame进行分组。DataFrameGro
    upBy
13 # 对象本身并不包含实际的数据, 而是存储了分组操作的信息, 允许你在分组上执行各种聚合操作。
14 print(g)
15
16 for i in g:
17     print(i)
18 for name, group in g:
19     print(name) # 输出组名
20     print("-"*100)
21     print(group) # 输出数据块
22     print(type(group))
23 # 选取任意类型的数据块
24 print(dict(list(df1.groupby(by='fruit'))['apple']))
25
26 # 以上都是分组 现在我想要计算数量呢?
27 # 就要使用聚合

```

函数名	描述
count	分组中非NAN值的数量
sum	非NAN值的和
mean	非NAN值的平均值
median	非NAN的中位数
std,var	标准差和方差
min,max	非NAN的最小值,最大值
prod	非NAN的乘积
first,last	非NAN值的第一个,最后一个

```

1  # 聚合
2  # 根据水果求价格的平均值
3  df1.groupby(by='fruit')['price'].mean()
4  # 语法糖
5  df1['price'].groupby(by='fruit').mean()# 报错了
6  df1['price'].groupby(df1['fruit']).mean()
7  # as_index 默认等于 True 行索引
8  df1.groupby(by='fruit',as_index=False)['price'].mean()
9
10 # 计算每一种水果的差值
11 # 自定义聚合函数
12 def diff(arr):
13     return arr.max()-arr.min()
14
15 # agg aggregate
16 df1.groupby(by='fruit')['price'].agg(diff)
17
18 # 根据国家和城市进行分组
19 df1.groupby(by=['fruit','color'])['price'].mean()
20 g1 = df1.groupby(by=['fruit','color'])['price'].mean()
21 print(type(g1))
22
23 g2 = df1.groupby(by='fruit')['price'].mean()
24 print(g2)
25
26 df1['price'].groupby(by = [df1['fruit'],df1['color']]).mean()
27 # 转换DataFrame
28 df1.groupby(by='fruit')[['price']].mean()
29
30
31 # 其他的分组形势
32 people = pd.DataFrame(np.random.randn(5,5),
33                        columns = ['a','b','c','d','e'],
34                        index = ['Sam','Make','Tony','Jim','Nana'])
35 people
36 # 通过字典进行分组
37 m = {'a':'red','b':'red','c':'blue','d':'blue','e':'red','f':'orange'}
38 people.groupby(m,axis=1).sum()
39 #通过Series进行
40 s1 = pd.Series(m)
41 s1
42 people.groupby(s1,axis=1).count()
43
44 # 通过函数进行分组
45 people.groupby(len).sum()
46
47 # 对每个分组同时应用多个聚合函数

```

```
48 agg_result = prople['Value'].agg(['sum', 'mean', 'count'])
49
50 # 为每个聚合函数指定自定义名称
51 agg_result = prople['Value'].agg(total='sum', avg='mean', count='count')
52
```

1.案例

作业:

- 1.分别读取以上四个csv文件,然后合并多个csv文件。
- 2.去重复数据,重新设置连续索引。
- 3.汽车分自动挡和手动挡,那么分别有多少辆?
- 4.哪个个城市二手车数量最多?
- 5.找出深圳市最多的二手车品牌?

龙仔

13.时间序列

为什么要学习pandas中的时间序列

时间序列(time series)数据是一种重要的结构化数据形式,应用于多个领域,包括金融学、经济学、生态学、神经科学、物理学等,在多个时间点观察或测量到的任何事物都可以形成一段时向序列,很多时间序列是固定须率的,也就是说,数据点是根据某种规律定期出现的(比如每15秒、每5分钟、每月出现一次),时间序列也可以是不定期的,没有固定的时间单位或单位之间的偏移量,时间序列数据的意义取决于具体的应用场景,主要有以下几种:

- 时间戳(timestamp),特定的时刻。
- 周定时期(period),2007年1月或2010年全年。
- 时间间隔(interval),由起始和结束时间截表示。时期(period)可以被看做间隔(interval)的特例。

1.日期和数据数据及工具

Python标准库包含用于日期(date)和时间(time)数据的数据类型，而且还有日历方面的功能，我们主要会用到datetime、time以及calendar模块，datetime.datetime(也可以简写为datetime)是用得最多的数据类型：

```
1  from datetime import datetime
2  now = datetime.now()
3  #YYYY表示年份，MM表示月份，DD表示日期，HH表示小时（24小时制）
4  #MM表示分钟，SS表示秒，sssss表示微秒。
5  now
6
7  now.year # 获取时刻的年份
8  now.month # 获取时刻的月份
9  now.day # 获取时刻的日
10
11 d = datetime(2011,1,7)-datetime(2008,6,24,8,15) # 计算时间差 计算器计算时间差
    16*3600-900
12 d
13 d.days # 天数
14 d.seconds # 秒数
15
16 # 除了减法也可以进行加号
17 from datetime import timedelta
18
19 start = datetime(2011,1,7)
20 # start+timedelta(12)*12
21 start+timedelta(12)*2 # 产生的是一个新的对象
```

类型	说明
date	以公历形式存储日历日期(年,月,日)
time	将时间存储为时,分,秒,毫秒
datetime	存储日期和时间
timedelta	表示两个datetime值之间的差(日,秒,毫秒)

字符串和datetime的相互转换

```

1 stamp = datetime(2011,1,3)
2 str(stamp)
3 # stamp.strftime("%Y-%m-%d %H %M %S")
4 # 使用strftime方法将日期对象格式化为指定的字符串格式。
5 # strftime是datetime对象的方法，用于将日期和时间格式化为字符串。
6 stamp.strftime("%Y-%m-%d %H %M %S") # 年 月 日 时 分 秒

```

代码	说明
%Y	4位数的年
%y	2位数的年
%m	2位数的月[01,02]
%d	2位数的日[01,31]
%H	时(24小时制)[00,23]
%I	时(12小时制)[01,12]
%M	2位数的分[00,59]
%S	秒[00,61](秒60和61用于闰秒)
%w	用整数表示的星期几[0（星期天），6]
%U	每年的第几周[00,53]。星期天被认为是每周的第一天，每年第一个星期天之前的那几天被认为是“第0周”
%W	每年的第几周[00,53]。星期一被认为是每周的第一天，每年第一个星期一之前的那几天被认为是“第0周”
%z	以+HHMM或-HHMM表示的UTC时区偏移量，如果时区为naive，则返回空字符串
%F	%Y%m-%d简写形式，例如2012-4-18
%D	%m/%d/%y简写形式，例如04/18/12

```

1 v = '2011-01-02' # 字符串 转 datetime
2 datetime.strptime(v, '%Y-%m-%d')
3 d = ['7/6/2011', '8/6/2012']
4 [datetime.strptime(i, '%Y-%m-%d') for i in d] # 报错
5 [datetime.strptime(i, '%m/%d/%Y') for i in d]
6
7 # 每次写这个格式定义很麻烦 就要使用第三方包中的方法
8 from dateutil.parser import parse # pandas中就帮我安装好了
9 parse('2011-01-03')
10
11 parse('6/12/2011') # 国际通用 错误的 日 月 年 打出来是 月 日 年
12
13 parse('6/12/2011', dayfirst=True) # 国际通用
14
15 # 那么以上都是单个的数据 在我们对数据(pandas)处理中都是成组的数据 如果处理呢?
16 import pandas as pd
17 d = ['2011-07-06 12:00:00', '2012-08-09 00:00:00']
18 pd.to_datetime(d) # 层级索引 还可以用来处理缺失值
19
20 idx = pd.to_datetime(d+[None])
21 idx
22 pd.isnull(idx)

```

2.时间序列基础


```

1  import numpy as np
2  import pandas as pd
3  from datetime import datetime
4
5  # 创建一个日期索引
6  dates = [datetime(2011, 1, 2), datetime(2011, 1, 5),
7           datetime(2011, 1, 7), datetime(2011, 1, 8),
8           datetime(2011, 1, 10), datetime(2011, 1, 12)]
9
10 # 创建一个时间序列
11 ts = pd.Series(np.random.randn(6), index=dates)
12
13 # 输出时间序列
14 ts
15
16 # 访问时间序列的索引
17 ts.index
18
19 # 时间序列的索引数据类型是numpy的datetime64，以纳秒为单位存储时间戳
20 ts.index.dtype
21
22 # 以每隔2个数据点为步长选择数据
23 ts[::2]
24
25 # 将时间序列与以每隔2个数据点为步长选择的时间序列相加
26 ts + ts[::2]
27
28 # 获取时间序列的第一个时间戳
29 s = ts.index[0]
30 s
31
32 # 通过时间戳索引选取数据
33 stamp = ts.index[2]
34 ts[stamp]
35
36 # 通过日期字符串选取数据
37 ts['1/10/2011']
38 ts['20110110']
39
40 # 使用切片选择数据，比如选择2001年的数据
41 ts1 = pd.Series(np.random.randn(1000), index=pd.date_range('1/1/2000', per
42                    iods=1000))
43 ts1['2001']
44
45 # 指定月份也可以
46 ts1['2001-05']

```

```

47 # 使用时间戳切片选取数据
48 ts[datetime(2011, 1, 7):]
49
50 # 使用日期范围进行切片选取数据
51 ts['1/6/2011':'1/11/2011'] # 注意，这是切片的副本，修改不会影响原始数据
52 a = ts['1/6/2011':'1/11/2011']
53 ts['1/6/2011':'1/11/2011'] = 1
54
55 # 使用truncate方法进行切片
56 ts.truncate(after='1/9/2011')
57
58 # 创建一个带有重复索引的时间序列
59 dates = pd.DatetimeIndex(['1/1/2000', '1/2/2000', '1/2/2000', '1/2/2000',
60 '1/3/2000'])
61 ts2 = pd.Series(np.arange(5), index=dates)
62
63 # 输出带有重复索引的时间序列
64 ts2
65
66 # 检查时间序列的索引是否唯一
67 ts2.index.is_unique # 如果返回False，则有重复索引
68
69 # 选择特定日期的数据
70 ts2['1/3/2000']
71
72 # 如果有重复索引，会返回一个时间序列
73 ts2['1/2/2000']
74
75 # 使用groupby方法对时间序列进行分组并计算均值。
76 ts2.groupby(level=0).mean()
77
78 # 使用groupby方法对时间序列进行分组并计算计数。
79 ts2.groupby(level=0).count()

```

3.生产一段时间范围

在Pandas中，你可以使用`pd.date_range`函数来生成一段时间范围，该函数用于创建日期时间索引。以下是如何使用`pd.date_range`函数来生成时间范围的示例：

```
pd.date_range(start=None, end=None, periods=None, freq="D")
```

```

1  import pandas as pd
2
3  # 生成一段时间范围，例如从 '2022-01-01' 到 '2022-01-10'
4  date_range = pd.date_range(start='2022-01-01', end='2022-01-10')
5  print(date_range) # d表示天
6
7  # 生成一段时间范围，从 '2022-01-01' 开始，共包含5天
8  date_range = pd.date_range(start='2022-01-01', periods=5)
9  print(date_range)
10
11 date_range = pd.date_range(end='2022-01-01', periods=5)
12 print(date_range)
13
14 date_range = pd.date_range(start='2022-01-01',end='2022-01-01', periods=5)
15 print(date_range)
16 # 因为在pd.date_range函数中，如果你同时指定了start、end和periods参数，
17 #它们之间必须满足一定的关系。start和end指定了时间范围的起始和结束日期，
18 #而periods指定了你希望在这个时间范围内生成多少个日期点。
19 # 具体来说，start和end之间的时间跨度必须与periods参数指定的日期数量相匹配。
20 #如果时间跨度不足以生成所需数量的日期点，将会导致错误
21 date_range = pd.date_range(start='2022-01-01',end='2022-01-01', freq="BM")
22 print(date_range)
23
24
25
26 # 生成一段时间范围，从 '2022-01-01' 开始，每隔2天取一个日期，共包含5个日期
27 date_range = pd.date_range(start='2022-01-01', periods=5, freq='2D')
28 print(date_range)
29
30 # 生成一段时间范围，从 '2022-01-01' 开始，每月最后一天，共包含5个日期
31 date_range = pd.date_range(start='2022-01-01', periods=5, freq='M')
32 print(date_range)
33

```

`pd.date_range` 函数的 `freq` 参数用于指定生成日期时间范围的频率或时间间隔。频率是一个字符串，用于表示生成的日期时间点之间的时间间隔。以下是一些常用的 `freq` 参数的示例：

1. 日频率：

- 'D' 或 'day'：每日。
- 'B'：每工作日（不包括周末）。
- 'W'：每周的指定日期（默认为周日）。

2. 月频率：

- 'M'：每月的最后一天。
- 'MS'：每月的第一天。

- **BM** 表示每月的最后一个工作日

- **BMS** 每月第一个工作日

3. 季度频率：

- **'Q'**：每季度的最后一天。

- **'QS'**：每季度的第一天。

4. 年频率：

- **'Y'**：每年的最后一天。

- **'YS'**：每年的第一天。

5. 小时、分钟和秒频率：

- **'H'**：每小时。

- **'T'** 或 **'min'**：每分钟。

- **'S'**：每秒。

- **'L'** 或 **'ms'**：每毫秒。

- **'U'**：每微秒。

你还可以将这些频率与数字结合使用，以指定多个时间间隔。例如，**'2D'** 表示每两天，**'3H'** 表示每三小时，**'5T'** 表示每五分钟，等等。

除了这些基本频率之外，Pandas还支持其他自定义频率，例如 **'W-MON'** 表示每周的星期一，**'BQ-JAN'** 表示每季度的一月份最后一个工作日，等等。

freq 参数非常灵活，允许你根据需要生成不同频率的日期时间点，适应各种时间序列数据的需求。

4. 频率和日期偏移量

在Pandas中，频率和日期偏移量是用于表示时间间隔的重要概念。它们允许你执行各种时间相关操作，如重采样、日期计算和日期偏移。以下是关于频率和日期偏移量的一些基本信息：

1. 频率：

- 频率是用于表示时间间隔的规则模式，通常以字符串形式表示，如**'D'**（表示日）、**'H'**（表示小时）、**'W'**（表示周）等。

- 频率字符串包括基本的时间单位，例如年（**'Y'**）、季度（**'Q'**）、月（**'M'**）、周（**'W'**）和日（**'D'**），以及复合频率，如小时（**'H'**）和分钟（**'T'**或**'min'**）等。

- 你可以使用频率字符串来定义时间范围或日期偏移。

2. 日期偏移量:

- 日期偏移量是用于表示时间间隔的对象，它们可以与频率一起使用。
- Pandas提供了一系列内置的日期偏移量，如`pd.DateOffset`、`pd.timedelta`等，用于表示特定时间间隔，如一天、一个小时等。
- 你可以使用日期偏移量来执行日期运算，如将日期向前或向后移动、计算两个日期之间的时间间隔等。

```
1 import pandas as pd
2
3 # 定义频率字符串
4 daily_freq = 'D' # 日频率
5 hourly_freq = 'H' # 小时频率
6
7 # 使用频率字符串生成时间范围
8 date_range = pd.date_range(start='2022-01-01', periods=5, freq=daily_freq)
9 print(date_range)
10
11 # 使用日期偏移量进行日期计算
12 date = pd.to_datetime('2022-01-01')
13 one_day = pd.DateOffset(days=1)
14 next_day = date + one_day
15 print(next_day)
16
17 # 创建一个日期时间索引，每隔2小时
18 hourly_range = pd.date_range(start='2022-01-01', periods=5, freq='2H')
19 print(hourly_range)
20
21 hourly_range = pd.date_range(start='2022-01-01', periods=5, freq='2H30min')
22 print(hourly_range)
23
24
```

5.移动数据

移动(shifting)指的是沿着时间轴将数据前移或后移。Series和DataFrame都有一个`shi`方法用于执行单纯的前移或后移操作，保持索引不变：

```
1 ts = pd.Series(np.random.randn(4), index=pd.date_range('1/1/2000', periods=4,
2 ts
3
4 ts.shift(2) # 往下移动
5
6 ts.shift(-2) # 上移动
7
8 ts.shift(2, freq='M')
9
```

在Pandas中，你可以使用**rolling**方法来执行数据的移动窗口操作。这在时间序列分析和数据平滑中非常有用，允许你计算滚动统计信息，如移动平均、移动总和等。以下是如何使用**rolling**方法来移动数据的示例：

```

1  import pandas as pd
2  import numpy as np
3
4  # 创建示例时间序列数据
5  data = {'date': pd.date_range(start='2022-01-01', periods=10),
6         'value': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]}
7  df = pd.DataFrame(data)
8
9  # 将日期列设置为索引
10 df.set_index('date', inplace=True)
11
12 # 使用rolling方法计算滚动均值
13 '''
14 这里的关键是window=3，它指定了滚动窗口的大小为3。这意味着在计算任何一个点的平均值之前，
15 需要有3个数据点在窗口内。这就是为什么前两个数据点的滚动平均值是NaN的原因：
16
17 对于数据集的第一个点（值为1），窗口内只有一个数据点，少于3个，所以无法计算平均值，结果是NaN。
18 对于第二个点（值为2），窗口内只有两个数据点（1和2），依然少于3个，所以平均值也是NaN。
19 只有从第三个数据点开始，窗口内才会有3个数据点（例如，对于第三个点，窗口内的数据是1, 2, 3），这时才能计算出一个有效的平均值。
20
21 这种处理方式在数据分析中很常见，特别是在处理时间序列数据时。NaN的出现是告诉我们，在那个特定的点，没有足够的数据来进行计算。
22
23 所以，在处理滚动统计数据时，通常需要考虑到序列开始处可能会有一些NaN值。希望这能帮助你更好地理解rolling方法的工作机制！
24 '''
25
26 rolling_mean = df['value'].rolling(window=3).mean()
27
28 # 输出滚动均值
29 print(rolling_mean)
30
31 # 使用rolling方法计算滚动总和
32 rolling_sum = df['value'].rolling(window=2).sum()
33
34 # 输出滚动总和
35 print(rolling_sum)

```

在上述示例中，我们首先创建一个包含时间序列数据的DataFrame，然后使用**rolling**方法计算滚动均值和滚动总和。在**rolling**方法中，**window**参数指定了滚动窗口的大小，用于计算统计信息。你可以根据需要调整**window**的大小以满足分析的要求。

滚动窗口操作可用于平滑时间序列数据，检测趋势和周期性，以及在数据处理中执行其他滚动操作。这对于时间序列分析和预测非常有帮助。

6.重采样

重采样：指的是将时间序列从一个频率转化为另一个频率进行处理的过程，将高频率数据转化为低频率数据为降采样，低频率转化为高频率为升采样

在Pandas中，重采样是一种用于将时间序列数据从一个频率转换为另一个频率的操作。这对于处理时间序列数据的不同时间分辨率、汇总数据和可视化非常有用。你可以使用**resample**方法来执行重采样操作。以下是一些关于如何在Pandas中进行重采样的示例


```

1  import pandas as pd
2  import numpy as np
3
4  # 创建一个包含100个行，每行一个随机数（范围10到50）的DataFrame，索引为从2017-01-01
   开始的100天。
5  t = pd.DataFrame(np.random.uniform(10,50,(100,1)), index=pd.date_range('20
   170101', periods=100))
6
7  # 输出DataFrame t 查看其内容
8  t
9
10 # 使用resample方法对t进行降采样，按月进行分组，然后计算每个月的平均值。
11 t.resample('M').mean()
12
13 # 使用resample方法对t进行降采样，按每10天进行分组，然后计算每个组的计数（即每10天内的
   数据点数量）。
14 t.resample('10D').count()
15
16 # 创建一个新的DataFrame，包含2行4列的随机数，索引为从2000-01-01开始每周三的两个日
   期，列名为不同城市。
17 frame = pd.DataFrame(np.random.randn(2,4),
18                       index=pd.date_range('1/1/2000', periods=2, freq='W-WE
   D'),
19                       columns=['上海', '北京', '深圳', '广州'])
20
21 # 使用resample方法对frame进行升采样，但不进行任何操作（仅显示结构）。
22 frame.resample('D')
23
24 # 使用resample方法对frame进行升采样，并使用asfreq()将其转换为高频率数据，但不进行填
   充。
25 frame.resample('D').asfreq()
26
27 # 使用resample方法对frame进行升采样，并使用ffill()向前填充缺失值。
28 frame.resample('D').ffill()
29
30 # 使用resample方法对frame进行升采样，并使用ffill()向前填充缺失值，但限制填充2次。
31 frame.resample('D').ffill(limit=2)
32
33 # 创建一个新的DataFrame，包含从2022-01-01开始的10天作为日期，以及一个0到9的整数序
   列。
34 data = {'date': pd.date_range(start='2022-01-01', periods=10, freq='D'),
35         'value': np.arange(10)}
36 df = pd.DataFrame(data)
37
38 # 将日期列设置为新DataFrame的索引。
39 df.set_index('date', inplace=True)
40

```

```
41 # 使用resample方法将df的数据从每日频率重采样为每两天的频率，并计算每组的均值。
42 resampled = df['value'].resample('2D').mean()
43
44 # 输出重采样的结果。
45 print(resampled)
46
```

在上述示例中，我们首先创建一个包含时间序列数据的DataFrame，然后使用**resample**方法将数据从每日频率重采样为每两天的频率，并计算每组的均值。在**resample**方法中，你可以使用不同的频率字符串来指定要重采样到的频率，如'D'（日）、'W'（周）、'M'（月）等。

另外，你可以在**resample**方法后附加聚合函数，如**mean()**、**sum()**、**max()**、**min()**等，以计算每个重采样组的统计信息。这对于在时间序列数据中计算统计信息非常有用。

重采样是一种重要的时间序列数据处理技术，可用于分析和可视化不同时间分辨率的数据，进行趋势分析，以及在时间序列数据的不同层次上执行各种操作。

1. 降采样 (Downsampling)

- **定义：**降采样是指将数据从高频率转换为低频率。比如，你有每天的数据，但你只需要每月的平均值。这个过程就是降采样。
- **目的：**降采样通常用于减少数据点数量，简化数据集，或者分析长期趋势。
- **例子：**如果你有每分钟的股市数据，将其转换为每小时或每天的平均值就是降采样。

2. 升采样 (Upsampling)

- **定义：**升采样是将数据从低频率转换为高频率。比如，你有每月的数据，但你需要每天的数据。这个过程就是升采样。
- **目的：**升采样通常用于使数据更加详细，用于预测或填补数据空缺。
- **挑战：**升采样的难点在于如何合理地估计或插入高频数据点。常用的方法包括插值、向前填充或向后填充。
- **例子：**如果你有每月的销售数据，但需要估计每天的销售情况，这就需要升采样并采用某种方法来估算每天的值。

7.时间序列案例

案例：股票价格分析

假设我们有一组包含公司股票价格的数据集，我们想要分析该股票随时间的表现。我们可以使用pandas来执行以下操作：

1. **解析日期：**从CSV文件读取数据时，确保日期列被解析成datetime对象。

2. **设置索引**：将日期列设置为DataFrame的索引，以便于基于时间的索引。
3. **重新采样**：将数据重新采样到不同的频率，如计算每月或每年的平均值，以观察长期趋势。
4. **滚动窗口**：计算滚动窗口统计数据，如7天滚动平均，以平滑短期波动。
5. **绘图**：使用DataFrame的绘图方法绘制股票价格随时间的变化，以及任何统计指标的变化。

这只是pandas在时间序列分析中应用的一个简单例子。pandas提供了非常丰富的方法来处理时间序列数据，包括但不限于时间点的索引、切片，以及时间区间和周期的转换等。